Foundations of RL Learning：理论与实验连贯梳理

作者：GitHub tequila28

项目链接：https://github.com/tequila28/Foundations-of-RL-Learning

项目初心：这个项目的出发点是从传统强化学习开始，系统理解 MDP、Bellman 方程、动态规划、Monte Carlo、TD Learning、函数近似和策略梯度等基础内容。传统 RL 是理解现代 LLM 强化学习的重要理论地基；无论是 RLHF、基于可验证奖励的 RLVR，还是更复杂的推理模型训练，都离不开“策略、奖励、价值、采样、优化、探索”这些基本概念。

本文档的前八章基于仓库代码版本 `ac68c8d041c23bc3985c4ea64fca02b940f3f419` 整理，第九章结合项目新增的 GSM8K、PPO 与 GRPO 代码。写作目标不是逐文件翻译代码，而是把九章内容串成一条学习路线：先建立环境和价值函数，再用动态规划求精确解，接着用采样学习，最后从函数近似和策略梯度走向 LLM 强化学习。

阅读时可以把这个项目看成一个逐层拆开的 RL 实验台。前几章假设环境模型已知，因此可以直接写出转移矩阵和 Bellman 方程；中间章节逐渐拿掉模型，改用经验轨迹估计价值；后几章再拿掉表格表示，改用特征和神经网络近似价值或策略。

0. 项目总体脉络

强化学习研究的是智能体如何在环境中连续决策。每一步智能体观察状态 `S_t`，选择动作 `A_t`，环境给出奖励 `R_{t+1}` 并转移到下一个状态 `S_{t+1}`。目标通常是最大化折扣回报：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

其中 `gamma in [0, 1)` 是折扣因子。`gamma` 越接近 1，算法越重视长期收益；越接近 0，算法越重视眼前奖励。

前八章以 5x5 GridWorld 为主线，第九章将状态和动作的概念迁移到 GSM8K 数学题与语言模型生成过程：

章节	主题	核心问题	是否需要环境模型
Chapter 1	基础概念与环境建模	什么是状态、动作、奖励、策略、价值	需要
Chapter 2	Bellman 方程	给定策略时如何计算价值	需要
Chapter 3	动态规划	如何在模型已知时求最优策略	需要
Chapter 4	Monte Carlo	没有模型时如何用完整轨迹学习	不需要
Chapter 5	随机近似	随机样本怎样逼近真实目标	不需要
Chapter 6	TD Learning	如何边采样边 bootstrap 学习	不需要

章节	主题	核心问题	是否需要环境模型
Chapter 7	价值函数近似	状态太多时如何用函数表示价值	不需要
Chapter 8	Policy Gradient	如何直接优化参数化策略	不需要
Chapter 9	LLM RL：PPO 与 GRPO	如何用最终答案奖励优化逐 token 生成的策略	不需要；使用采样回答与规则奖励

一个值得先说明的项目细节是：GridWorld 中的目标格子不是 episode 终止状态。到达目标格会得到正奖励，但智能体仍可继续行动；如果在目标格执行 `stay`，会反复得到正奖励。因此这里更像一个折扣持续任务，而不是“走到终点就结束”的迷宫任务。在 `gamma=0.9`、目标奖励为 `1` 的章节中，目标格持续停留的理论价值接近：

$$1 + 0.9 + 0.9^2 + \cdots = \frac{1}{1 - 0.9} = 10$$

这个设定会影响后面所有图的解释：某个策略价值高，不只是因为它“找到目标”，还因为它能不能持续留在高奖励状态附近。

0.1 从监督学习到强化学习：为什么这里要多一层“环境”

监督学习的数据通常是固定的 `(x, y)`，模型只需要学习从输入到标签的映射。强化学习多了一个困难：智能体选择动作以后，会改变自己之后看到的数据。也就是说，数据分布不是提前固定好的，而是由策略和环境共同生成的。

在 GridWorld 里，如果策略总是往右走，智能体会频繁访问右侧边界附近的状态；如果策略会主动寻找目标格，它就会更多访问目标格周围的状态。于是同一个算法在不同探索策略下看到的经验不同，最终学到的价值也会不同。这就是为什么后面 MC、TD、函数近似实验都要反复讨论“采样策略”和“目标策略”。

强化学习中的几个核心对象可以这样对应：

数学对象	直观含义	项目中的表现
状态 `s`	智能体当前所处情形	5x5 网格中的一个格子
动作 `a`	智能体可以做的选择	上、右、下、左、停留
奖励 `r`	环境对动作结果的即时反馈	目标奖励、撞墙惩罚、禁止格惩罚
策略 `pi(a|s)`	每个状态下如何选动作	
转移 `P(s'|s,a)`	动作如何改变状态	
价值 `v(s)`	从状态出发的长期好坏	可视化图中的颜色和数字

这套对应关系是整份项目的底层语言。后面无论算法名字变成 DP、MC、TD、Q-learning 还是 Actor-Critic，本质上都在估计或优化这些对象。

1. Chapter 1：基础概念与 GridWorld 环境

1.1 理论知识点

一个有限 Markov Decision Process 可以写成五元组：

$$\text{MDP} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

其中：

- `S` 是有限状态集合；
- `A` 是有限动作集合；
- `P(s' | s, a)` 是状态转移概率；
- `R(s, a)` 是在状态 `s` 执行动作 `a` 的即时奖励；
- `gamma` 是折扣因子。

策略 `pi(a | s)` 描述智能体在每个状态下选择动作的概率。给定策略后，状态价值函数定义为：

$$v_\pi(s) = \mathbb{E}_\pi\left[G_t \mid S_t = s\right]$$

动作价值函数定义为：

$$q_\pi(s,a) = \mathbb{E}_\pi\left[G_t \mid S_t = s, A_t = a\right]$$

二者的关系是：

$$v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s,a)$$

$$q_\pi(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v_\pi(s')$$

从这两个式子可以看出，`v` 和 `q` 不是两个互不相干的概念。`q_pi(s,a)` 先固定第一步动作，问“如果我在 `s` 做 `a`，然后继续按策略 `pi` 行动，长期回报是多少”；`v_pi(s)` 则是不固定第一步动作，而是按策略对所有动作价值做加权平均。

如果我们想找最优策略，就会定义最优价值：

$$v^*(s) = \max_\pi v_\pi(s), \qquad q^*(s,a) = \max_\pi q_\pi(s,a)$$

最优策略可以从最优动作价值中直接得到：

$$\pi^*(s) \in \arg\max_a q^*(s,a)$$

这解释了为什么后面很多控制算法都学习 `Q(s,a)`：一旦有了动作价值，不需要知道环境模型，也能通过 `argmax` 选择动作。

1.2 项目实现

第 1 章实现了一个 5x5 网格世界。状态编号按照 `row * 5 + col` 展开，因此左上角是状态 0，右下角是状态 24。动作集合为：

```
up, right, down, left, stay
```

环境默认配置如下：

项目	设置
网格大小	5x5
状态数	25
动作数	5
禁止格	6, 7, 12, 16, 18, 21
目标格	17
折扣因子	0.9
撞墙奖励	-1
进入禁止格奖励	-1
进入目标格奖励	1
普通转移奖励	0

这里的“禁止格”并不是不可进入的墙，而是进入后得到惩罚的格子。撞到边界时智能体留在原状态并受到惩罚。

1.2.1 代码如何从零构建网格环境

第 1 章最重要的代码是环境类。它做的事情可以按四步理解。

第一步是把二维网格压平成一维状态编号。代码中的状态不是 `(row, col)` 元组，而是整数：

```
state = row * size + col
row = state // size
col = state % size
```

这样做的好处是，价值函数可以直接写成长度为 25 的向量：

```
V = [V(0), V(1), ..., V(24)]
```

状态转移矩阵也可以写成普通 NumPy 数组，而不用处理复杂对象。

第二步是定义动作到坐标变化的映射：

```
up    -> row - 1, col
right -> row, col + 1
down  -> row + 1, col
left  -> row, col - 1
stay  -> row, col
```

每个动作先尝试给出一个候选下一格。如果候选位置越界，下一状态就保持原状态，并给撞墙惩罚 `-1`。如果没有越界，再检查候选格是不是禁止格或目标格，从而决定奖励。

第三步是为每个 `(state, action)` 填入下一状态和奖励。由于当前环境是确定性的，一个状态动作对只有一个后继状态。所以转移张量满足：

$$P[s, a, \text{next_state}] = 1, \qquad P[s, a, \text{other}] = 0$$

$$R[s, a] = \text{reward}$$

如果环境是随机的，例如执行 `up` 时有 80% 概率向上、10% 概率向左、10% 概率向右，那么同一行 `P[s,a,:]` 会有多个非零元素。但本项目的 GridWorld 没有这种随机滑动，因此转移矩阵更容易检查和可视化。

第四步是把策略也转换成矩阵。一个策略矩阵的形状是：

```
policy.shape = (num_states, num_actions)
```

其中 `policy[s,a]` 表示在状态 `s` 选择动作 `a` 的概率。给定策略以后，代码会把原始环境模型 `P[s,a,s']` 和 `R[s,a]` 压缩成策略诱导的 Markov chain：

$$P_\pi[s, s'] = \sum_a \pi(a \mid s) P[s, a, s']$$

$$r_\pi[s] = \sum_a \pi(a \mid s) R[s, a]$$

这样，一个有动作的 MDP 就暂时变成了一个没有决策的 Markov Reward Process。第 2 章的 Bellman 方程正是建立在 `P_pi` 和 `r_pi` 上。

下面这段伪代码概括了环境构建过程：

```
for state in all_states:
    row, col = decode(state)
    for action in all_actions:
        next_row, next_col = move(row, col, action)

        if out_of_bounds(next_row, next_col):
            next_state = state
            reward = r_bound
        else:
            next_state = encode(next_row, next_col)
            if next_state in forbidden:
                reward = r_forbid
            elif next_state in target:
                reward = r_target
            else:
                reward = r_default

        P[state, action, next_state] = 1
        R[state, action] = reward
```

这一段代码其实就是把 MDP 五元组里的 `S, A, P, R, gamma` 具体化。后面所有算法都没有脱离这套基础数据结构：DP 直接使用 `P` 和 `R`；MC 和 TD 虽然不显式求解矩阵，但每一步采样 transition 时，本质上仍然在调用这套环境动力学。

1.2.2 为什么目标格不终止会改变价值

很多迷宫例子默认“到达终点 episode 结束”，此时目标奖励只拿一次。这个项目不是这样。目标格状态 17 执行 `stay` 后仍然回到 17，并再次获得目标奖励。因此：

$$R(17, \text{stay}) = 1, \qquad P(17 \mid 17, \text{stay}) = 1$$

代入 Bellman 方程：

$$V(17) = 1 + 0.9V(17)$$

$$V(17) = 10$$

这就是为什么动态规划章节里目标格最优价值接近 10，而不是 1。到了第 8 章，目标奖励被设为 10，同样逻辑下如果持续停留，折扣无限价值会接近 100；但第 8 章的 episode 最长只有 20 步，所以训练曲线里的单轮未折扣回报上界约为 200。

代码会构造完整的转移张量 `P[s, a, s']` 和奖励矩阵 `R[s, a]`。因为环境是确定性的，每个 `(s, a)` 只会指向一个下一状态。第 1 章随后用一个随机策略生成策略矩阵，并通过线性方程直接计算该策略的价值函数。

1.2.3 这一章做了什么，为什么要这样做

第 1 章的任务不是训练一个智能体，而是先搭建后续所有算法共享的“世界”。它完成了三件基础工作：定义状态空间、定义动作空间、定义每个动作带来的状态转移和奖励。没有这一步，后面的 Bellman 方程就没有 `P` 和 `R`，动态规划也没有可以计算的模型，MC 和 TD 也没有可以交互采样的环境。

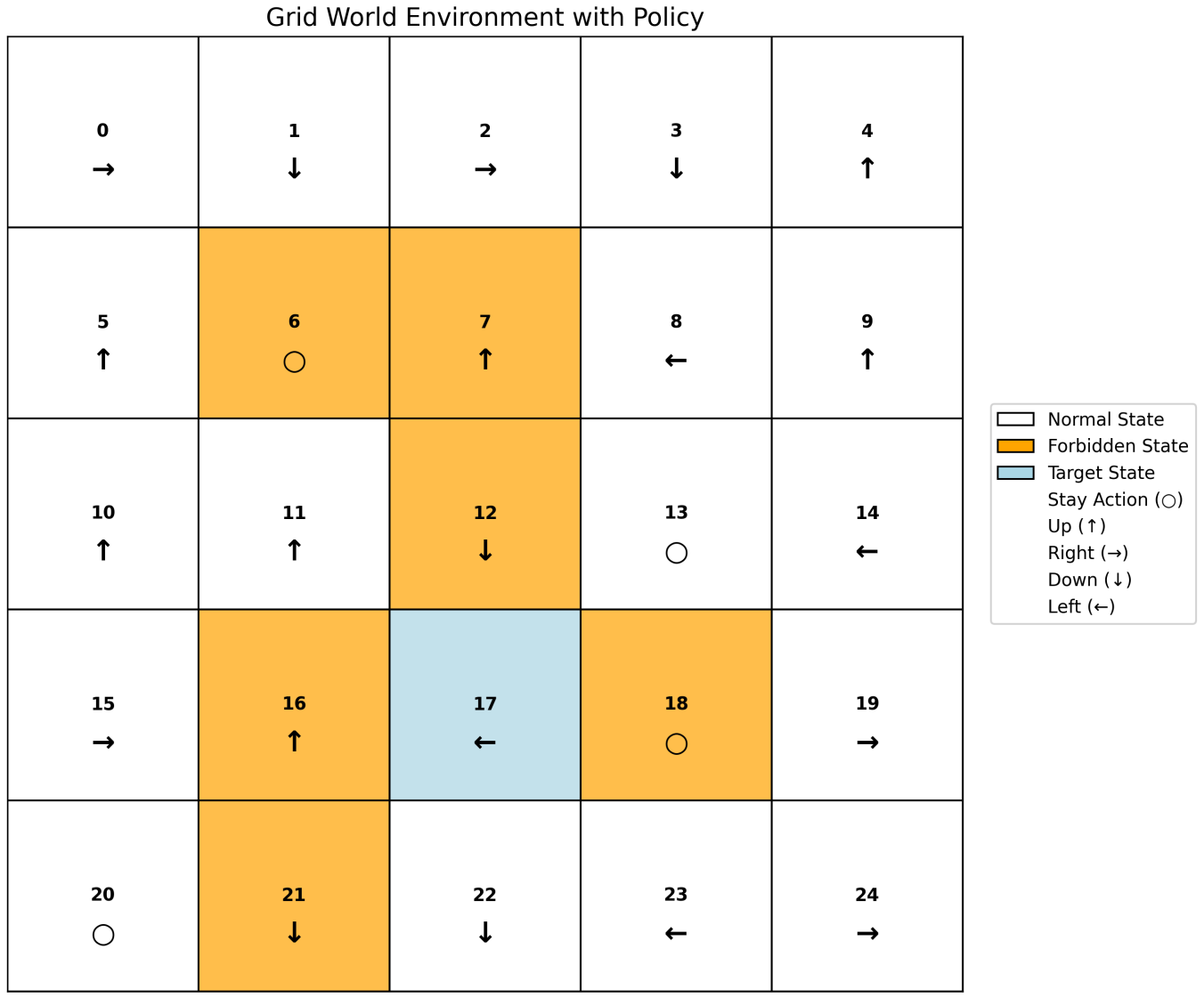
这一章还把策略从直觉概念变成了具体数组。很多初学者会把策略理解成“往目标走的规则”，但在强化学习里，策略首先是一个概率分布。即使一个策略看起来是随机的，只要它给出了每个状态下五个动作的概率，就可以被写成矩阵、参与期望计算、生成轨迹并被可视化。

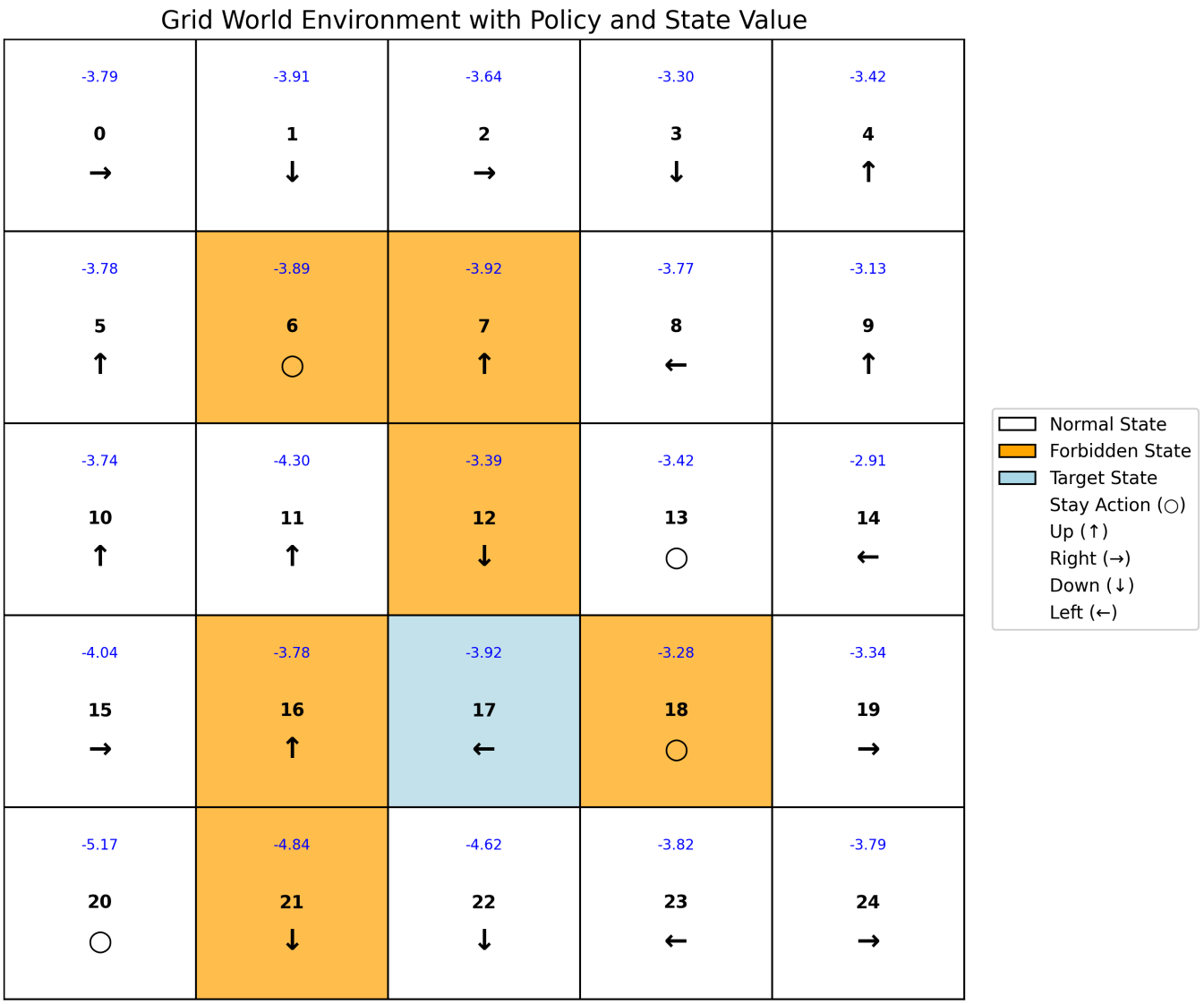
本章实验做的事情可以概括为：

步骤	具体内容	为什么做
构建 GridWorld	生成 25 个状态、5 个动作、奖励规则和转移模型	给后续算法提供统一环境
构建随机策略	为每个状态分配动作概率	展示策略可以被矩阵化
计算策略价值	根据策略诱导的转移模型求 `V`	让“长期回报”从公式变成图
可视化策略和值	用箭头和颜色展示策略行为	方便检查环境和价值是否符合直觉

这一章之所以重要，是因为它固定了实验基准。后面所有算法看起来不同，但它们都在同一个 GridWorld 上回答同一个问题：怎样让智能体更倾向于避开惩罚、靠近目标，并在目标附近获得长期回报。

1.3 实验结果解读


Grid World Environment with Policy
0 →
1 ↓
2 →
3 ↓
4 ↑
5 ↑
6 ○
7 ↑
8 ←
9 ↑
10 ↑
11 ↑
12 ↓
13 ○
14 ←
15 →
16 ↑
17 ←
18 ○
19 →
20 ○
21 ↓
22 ↓
23 ←
24 →
Normal State
Forbidden State
Target State
Stay Action (○)
Up (↑)
Right (→)
Down (↓)
Left (←)



第一张图展示了每个状态下的动作概率，第二张图展示对应价值。由于策略是随机的，智能体经常会撞墙、进入惩罚格，或者离开目标格，所以价值分布不会像最优策略那样集中向目标格抬升。

第 1 章的实验意义在于：它把抽象的 MDP 对象落到了可计算的数据结构上。后续章节的 Bellman 方程、动态规划和 TD 更新都复用这套环境表示。

2. Chapter 2：Bellman 方程

2.1 理论知识点

Bellman 方程是强化学习的核心递归关系。给定策略 `pi`，状态价值满足：

$$v_\pi(s) = r_\pi(s) + \gamma \sum_{s'} P_\pi(s' \mid s) v_\pi(s')$$

其中：

$$r_\pi(s) = \sum_a \pi(a \mid s) R(s,a)$$

$$P_\pi(s' \mid s) = \sum_a \pi(a \mid s) P(s' \mid s,a)$$

写成矩阵形式：

$$v_\pi = r_\pi + \gamma P_\pi v_\pi$$

移项得到闭式解：

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi$$

只要 `gamma < 1`，Bellman 算子是压缩映射，价值函数存在唯一解。实际计算时未必显式求逆，直接解线性方程通常更稳定：

$$(I - \gamma P_\pi) v_\pi = r_\pi$$

除了闭式解，也可以用迭代法：

$$v_{k+1} = r_\pi + \gamma P_\pi v_k$$

迭代法从任意初始 `v_0` 出发，反复应用 Bellman expectation operator，最终收敛到 `v_pi`。

这里有两类 Bellman 方程需要区分。第一类是 Bellman expectation equation，它评价一个已经给定的策略：

$$v_\pi(s) = \sum_a \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v_\pi(s') \right]$$

第二类是 Bellman optimality equation，它直接描述最优价值：

$$v^*(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v^*(s') \right]$$

两者只差一个地方：前者对动作按策略求期望，后者对动作取最大值。这个差别非常关键。第 2 章只做“策略评估”，所以用 expectation equation；第 3 章要找最优策略，所以开始使用 optimality equation。

从算法角度看，Bellman 方程也是一种“自洽条件”。如果一个价值函数是真的，那么它今天的价值必须等于“今天拿到的奖励 + 明天价值的折扣期望”。DP、TD、Q-learning 的更新都可以看成是在不断修正当前价值，使它越来越接近这个自洽条件。

2.2 项目实现

第 2 章比较了两类策略评估方法：

- 闭式求解：调用线性方程求解器；
- 迭代求解：从零价值开始做 Bellman 更新，直到最大变化量低于阈值。

实验设置了四种策略，包括两个手工设计的策略、一个偏差较大的策略和一个随机生成的策略。代码里的最后一种随机策略是每个状态随机生成正数后归一化，不是严格的五个动作等概率策略。阅读图时要把它理解为“随机概率策略”，而不是“均匀随机策略”。

2.2.1 这一章实现了什么，为什么要这样做

第 2 章的核心工作是“给定策略，计算它到底好不好”。第 1 章已经能生成一个策略，也能得到环境模型；第 2 章把这两个对象组合起来，求出每个状态在该策略下的长期价值。

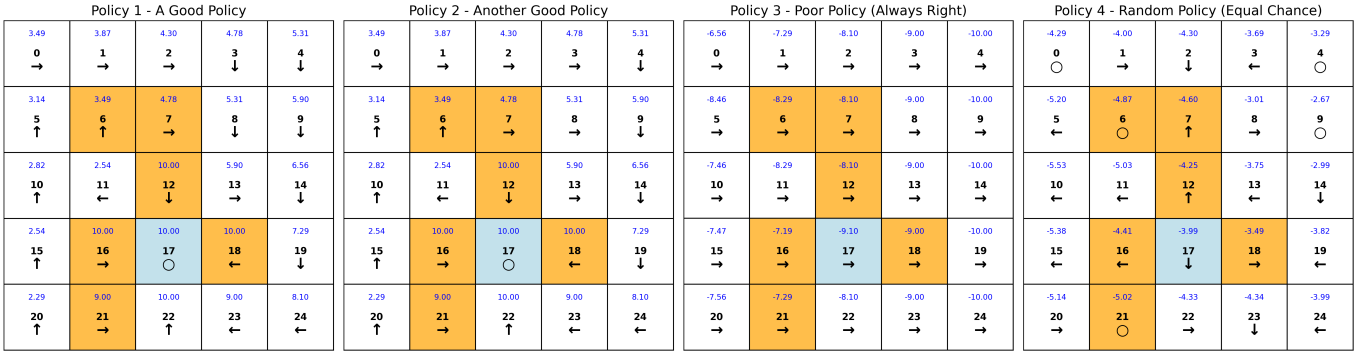
本章实现的内容包括：

模块	做的事情	对应理论
策略诱导模型	把 `P[s,a,s']` 和 `R[s,a]` 按 `pi(a	s)` 加权
闭式求解	解 `(I - gamma P_pi)v = r_pi`	Bellman 方程矩阵形式
迭代求解	重复 `v <- r + gamma P v`	Bellman 算子收敛
多策略对比	比较不同策略的价值图	策略决定长期回报

为什么要做闭式解和迭代解的对比？因为它们代表两种理解 Bellman 方程的方式。闭式解说明价值函数是一个线性方程组的解，理论上清楚；迭代解说明价值可以通过不断备份后继状态逐步传播，算法上更接近后面的 DP 和 TD。对于小规模 GridWorld，两者都能用；对于大规模问题，显式线性求解通常不可行，迭代思想就更重要。

本章还说明了一个关键事实：策略不同，价值函数会完全不同。即使环境没有变，动作概率稍有变化，长期访问惩罚格或目标格的频率就会变化，最终价值图也会变化。这为第 3 章的“策略改进”埋下伏笔：既然能评价策略，就可以尝试系统地让策略变好。

2.3 实验结果解读





两张图分别展示闭式求解和迭代求解得到的策略价值。项目中我额外做了一个小规模数值核查：同一均匀策略下，两种方法的最大差异约为 `9.2e-6`。这说明在当前阈值下，迭代 Bellman 方程已经非常接近线性方程精确解。

第 2 章承接第 1 章：只要能从策略得到 `P_pi` 和 `r_pi`，就能算出该策略在环境中的长期价值。下一章要解决的问题是：不仅评估一个策略，还要找到最优策略。

3. Chapter 3：Policy Iteration、Value Iteration 与 Truncated Policy Iteration

3.1 理论知识点

动态规划方法假设环境模型 P 和 R 已知，因此可以直接在状态空间上更新价值函数。

策略迭代由两步交替组成：

$$\text{Policy Evaluation:} \quad v_\pi \leftarrow r_\pi + \gamma P_\pi v_\pi$$

$$\text{Policy Improvement:} \quad \pi_{\text{new}}(s) \leftarrow \arg\max_a \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v_\pi(s') \right]$$

策略改进定理保证，如果对所有状态都有：

$$q_\pi(s, \pi_{\text{new}}(s)) \geq v_\pi(s)$$

那么新策略不会比旧策略差。反复评估和改进，最终会得到最优策略。

价值迭代把"完整策略评估"压缩成一次最优 Bellman 更新：

$$v_{k+1}(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v_k(s') \right]$$

收敛后再取贪婪策略：

$$\pi^*(s) = \arg\max_a \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v^*(s') \right]$$

截断策略迭代介于两者之间：每次策略评估只做固定次数，然后立刻策略改进。它展示了一个很重要的思想：很多 RL 算法都不需要把每个中间子问题求到完全精确，只要更新方向足够好，也能逐步逼近最优策略。

这三种算法的共同点是都在应用 Bellman backup。所谓 backup，就是用后继状态的价值反过来更新当前状态的价值：

```
current estimate -> look one step ahead -> backup new estimate
```

Policy Iteration 的 backup 服务于当前策略，Value Iteration 的 backup 直接选择最优动作，Truncated Policy Iteration 则在两者之间折中。理解 backup 以后，后面的 TD 会自然很多：TD 只是把"已知模型下对所有后继求期望"换成了"采样到哪个后继，就先用哪个后继更新一次"。

3.2 项目实现

第 3 章实现了三种算法：

算法	项目中的做法
Policy Iteration	对当前策略做迭代评估，收敛后贪婪改进
Value Iteration	对每个状态做最优 Bellman 更新，最后提取策略
Truncated Policy Iteration	每轮只做 x 次策略评估，然后改进

价值迭代和策略评估采用逐状态原地更新，因此严格说不是完全同步更新。对当前小规模确定性 GridWorld 来说，这不会改变算法含义，但会影响迭代次数和收敛曲线的细节。

3.2.1 这一章实现了什么，为什么要这样做

第 3 章从“评价一个策略”进入“寻找最优策略”。这一步是强化学习控制问题的核心：智能体不只是想知道某个行为方式有多好，而是要找到更好的行为方式。

本章围绕同一个 Bellman backup 实现三种控制方法：

方法	本章具体做法	为什么做
Policy Iteration	先把当前策略评估到收敛，再贪婪改进	展示“评估 + 改进”的完整控制闭环
Value Iteration	每次直接用最大动作价值更新状态价值	展示不必完整评估中间策略也能求最优
Truncated PI	每轮只评估固定次数，再做策略改进	展示计算量和评估精度之间的折中

这章实验的价值在于，它把“最优性”变成了可以计算的过程。策略迭代像一个谨慎流程：先充分评估当前策略，再决定怎么改。价值迭代更直接：每次都朝当前 Bellman 最优目标更新。TPI 则介于两者之间，体现了很多实际 RL 算法的味道：不用每一步都做到完全精确，只要整体方向能推动策略变好。

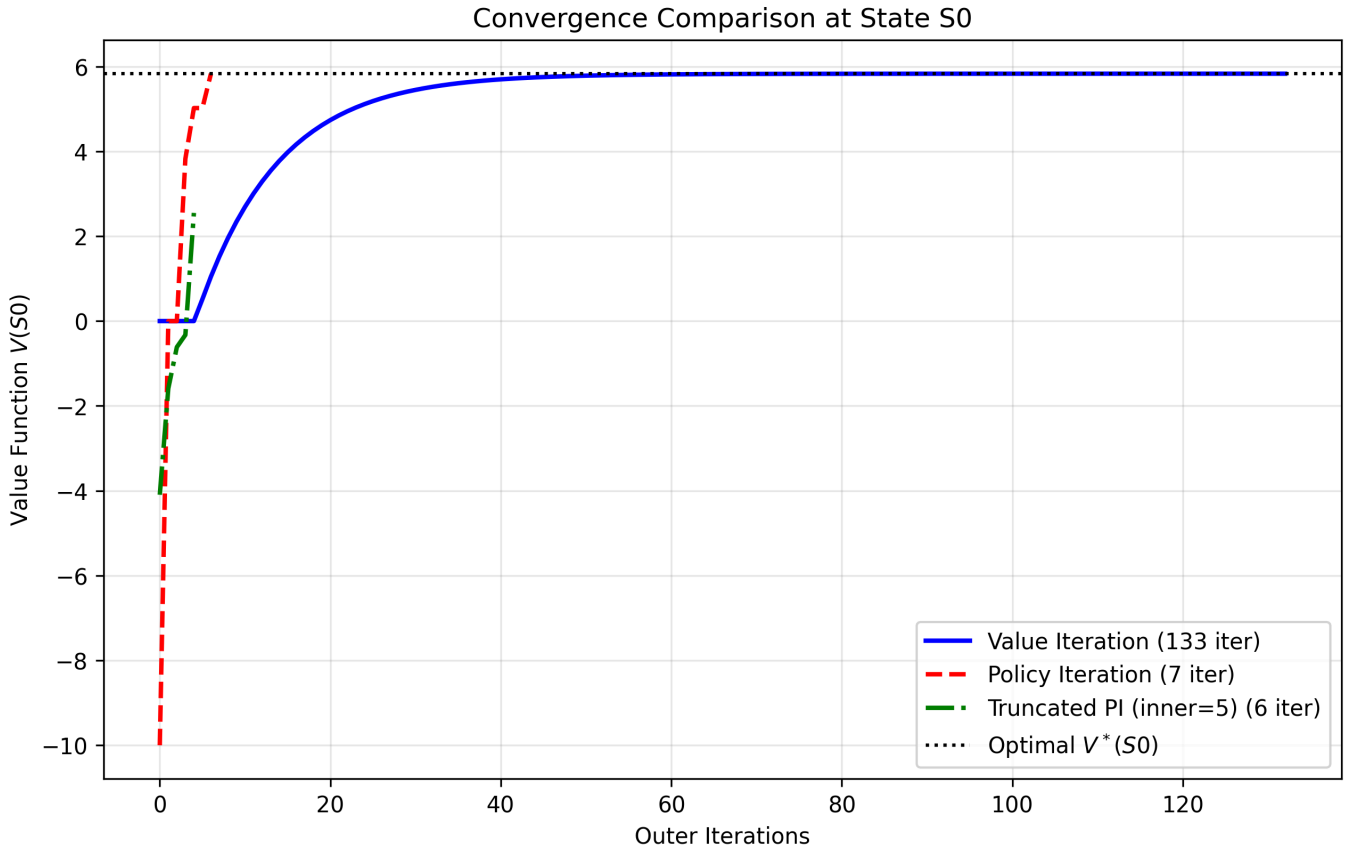
本章为什么放在 MC 和 TD 之前？因为 DP 是最干净的版本：模型已知、没有采样噪声、价值传播完全由 Bellman 方程控制。先看 DP，可以知道“如果信息完整，正确答案应该怎样被求出来”。后面 model-free 方法学到的东西，本质上都是在没有完整模型时近似这个过程。

我额外核查了几个关键数值：

检查项	结果
Value Iteration 与 Policy Iteration 的最大价值差	`6.84e-6`
目标格 `stay` 的下一状态	仍为目标格 17
目标格 `stay` 的即时奖励	`1.0`
`gamma=0.9` 下目标格最优价值	约 `9.99999`

这些结果与前面提到的“目标格不是终止状态”一致。

3.3 实验结果解读



Value Iteration (133 iters)

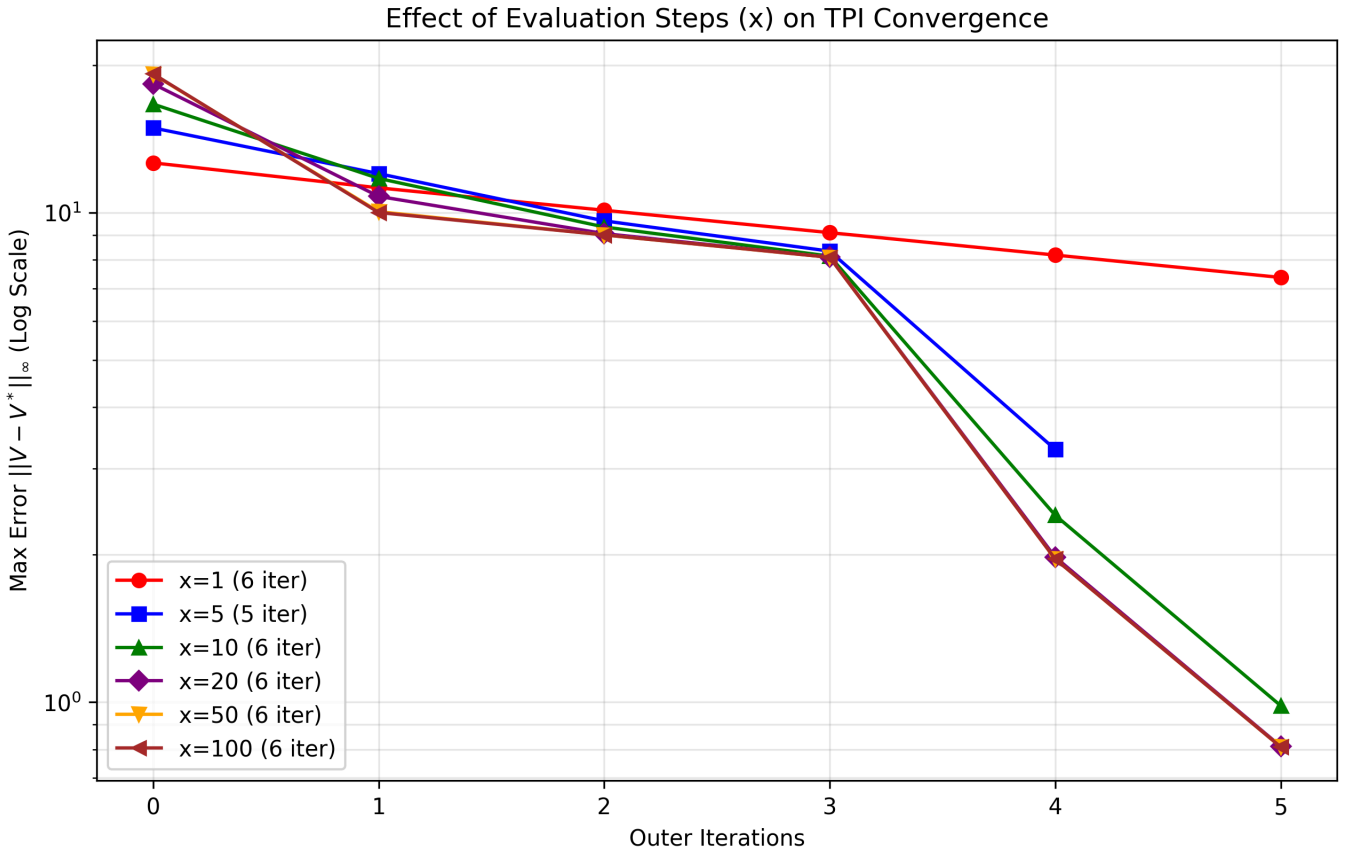
5.83 0 ↓	5.58 1 →	6.20 2 ↓	6.48 3 ↓	5.83 4 ↓
6.48 5 ↓	7.20 6 ↓	8.00 7 ↓	7.20 8 ↓	6.48 9 ↓
7.20 10 →	8.00 11 →	10.00 12 ↓	8.00 13 ↓	7.20 14 ↓
8.00 15 →	10.00 16 →	10.00 17 ○	10.00 18 ←	8.00 19 ←
7.20 20 ↑	9.00 21 →	10.00 22 ↑	9.00 23 ←	8.10 24 ←

Policy Iteration (7 iters)

5.83 0 ↓	5.58 1 →	6.20 2 ↓	6.48 3 ↓	5.83 4 ↓
6.48 5 ↓	7.20 6 ↓	8.00 7 ↓	7.20 8 ↓	6.48 9 ↓
7.20 10 →	8.00 11 →	10.00 12 ↓	8.00 13 ↓	7.20 14 ↓
8.00 15 →	10.00 16 →	10.00 17 ○	10.00 18 ←	8.00 19 ←
7.20 20 ↑	9.00 21 →	10.00 22 ↑	9.00 23 ←	8.10 24 ←

Truncated PI (inner=5) (6 iters)

4.37 0 ↓	4.12 1 →	4.74 2 ↓	5.17 3 ↓	4.65 4 ↓
5.02 5 ↓	5.74 6 ↓	6.54 7 ↓	5.89 8 ↓	5.30 9 ↓
5.74 10 →	6.54 11 →	8.54 12 ↓	6.69 13 ↓	6.02 14 ↓
6.54 15 →	8.54 16 →	8.54 17 ○	8.69 18 ←	6.82 19 ←
5.79 20 →	7.69 21 →	8.69 22 ↑	7.82 23 ←	7.04 24 ←



收敛曲线说明，价值迭代通常能较快把价值推向最优附近；策略迭代每轮计算更重，但策略稳定后能直接停下。TPI 的敏感性图展示了评估步数 `x` 的影响：`x` 太小会导致策略评估太粗糙，价值误差明显；`x` 增大后，TPI 更接近完整策略迭代。

额外核查中，TPI 最终价值误差随 `x` 增大明显下降：

x	最终价值误差
1	`6.6309`
5	`1.4596`
10	`0.0605`
20	`0.00028`
50	`8.21e-6`
100	`8.21e-6`

这一章的核心结论是：在模型已知时，Bellman 最优方程可以直接带来最优策略；不同 DP 算法是在“每轮计算量”和“收敛轮数”之间做权衡。

4. Chapter 4：Monte Carlo 方法

4.1 理论知识点

动态规划需要知道环境模型，但真实任务中通常无法直接得到 `P(s'|s,a)` 和 `R(s,a)`。Monte Carlo 方法用采样轨迹估计回报：

$$Q(s,a) \leftarrow \text{Average}\left(G_t \mid S_t = s, A_t = a\right)$$

如果一条 episode 为：

```
S_0, A_0, R_1, S_1, A_1, R_2, ...
```

那么从时间 `t` 开始的回报是：

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T$$

MC 的优点是不需要模型，也不需要 bootstrap；缺点是必须等一段轨迹结束后才能更新，方差较大，而且在持续任务中通常要人为截断轨迹。

探索起始 Monte Carlo control 使用随机的起始 `(s,a)`，保证状态动作对有机会被访问。epsilon-greedy control 则在每一步以 `epsilon` 的概率探索，以 `1-epsilon` 的概率选择当前贪婪动作。

MC 方法有两个常见版本：first-visit 和 every-visit。first-visit 只在一个 episode 中第一次访问某个状态或状态动作对时更新；every-visit 则每次访问都更新。样本足够多时，两者都可以收敛到正确期望，但有限样本下方差和更新次数不同。

在控制问题中，MC 还必须处理探索与利用的矛盾。如果总是选择当前估计最好的动作，早期偶然误差可能让智能体长期忽略更好的动作；如果一直随机探索，策略又很难稳定到高回报行为。epsilon-greedy 正是一个简单折中：保留少量探索，同时让策略逐渐偏向当前最优动作。

4.2 项目实现

第 4 章实现了三类 MC 控制实验：

方法	样本生成方式	更新特点
Basic MC	遍历所有状态动作对生成短轨迹	轨迹长度 10
Exploring Starts	每次随机起始状态动作	10000 条轨迹
Epsilon-Greedy MC	按 epsilon-greedy 策略持续采样	every-visit 更新

有一个细节需要注意：Basic MC 和 Exploring Starts 的代码从轨迹末尾反向遍历，并用 `visited` 去重。这样选到的是反向遍历时的首次访问，也就是正向时间上的最后一次访问，不是教材中常见的 first-visit MC。这个实现仍然是在用经验回报估计 `Q(s,a)`，但报告中不宜把它严格称为 first-visit MC。

另外，由于项目中的 GridWorld 是持续任务，第 4 章把轨迹截断为有限长度并令尾部回报为 0。这会带来截断偏差。以 `gamma=0.9`、单步奖励绝对值上界为 1、轨迹长度 10 为例，未计入尾部的理论上界约为：

$$\frac{\gamma^{10}}{1-\gamma} = \frac{0.9^{10}}{0.1} \approx 3.49$$

因此 MC 图中的价值不一定接近无限折扣任务的精确价值。

4.2.1 这一章实现了什么，为什么要这样做

第 4 章开始进入 model-free learning。前面几章都直接使用环境模型，知道每个动作一定会转移到哪里；Monte Carlo 方法则假设我们不知道模型，只能通过一次次试走来估计价值。

本章实现的重点是“用实际回报替代 Bellman 期望”。如果一个状态动作对 `(s,a)` 在多条轨迹中出现过很多次，那么每次出现后得到的折扣回报都可以看成一次样本。对这些样本求平均，就能估计 `Q(s,a)`。项目进一步在估计 `Q` 后取贪婪动作，于是 MC 不只是在预测价值，也在做控制。

本章三组实验各自承担不同作用：

实验	做了什么	为什么做
Basic MC	系统遍历状态动作对，生成固定长度轨迹	保证每个 `(s,a)` 至少有机会被估计
Exploring Starts	每条轨迹随机选择初始 `(s,a)`	用随机起点改善探索覆盖
Epsilon-Greedy MC	在轨迹内部持续探索和利用	更接近实际控制算法的采样方式

为什么这一章要强调探索？因为没有模型时，智能体只知道自己经历过的东西。如果从来没有尝试过某个动作，就无法知道它是否更好。MC control 的一个主要难点不是公式复杂，而是怎样保证所有重要状态动作对都能被充分访问。

这一章也让我们第一次看到采样算法的缺点：轨迹越短，长期价值越难估；轨迹越长，方差越大，学习也越慢。这个矛盾正是第 6 章 TD Learning 要解决的问题。

4.3 实验结果解读

MC Basic
Final Policy

2.00 0 ↓	1.74 1 →	2.17 2 ↓	2.52 3 ↓	2.03 4 ↓
2.44 5 ↓	3.30 6 ↓	3.75 7 ↓	2.94 8 ↓	2.53 9 ←
2.98 10 →	3.66 11 →	4.95 12 ↓	3.51 13 ↓	3.20 14 ←
3.69 15 →	5.20 16 →	1.00 17 ○	5.13 18 ←	3.88 19 ←
3.38 20 ↑	5.12 21 →	5.45 22 ↑	4.95 23 ←	4.23 24 ←

MC Exploring Starts
Final Policy

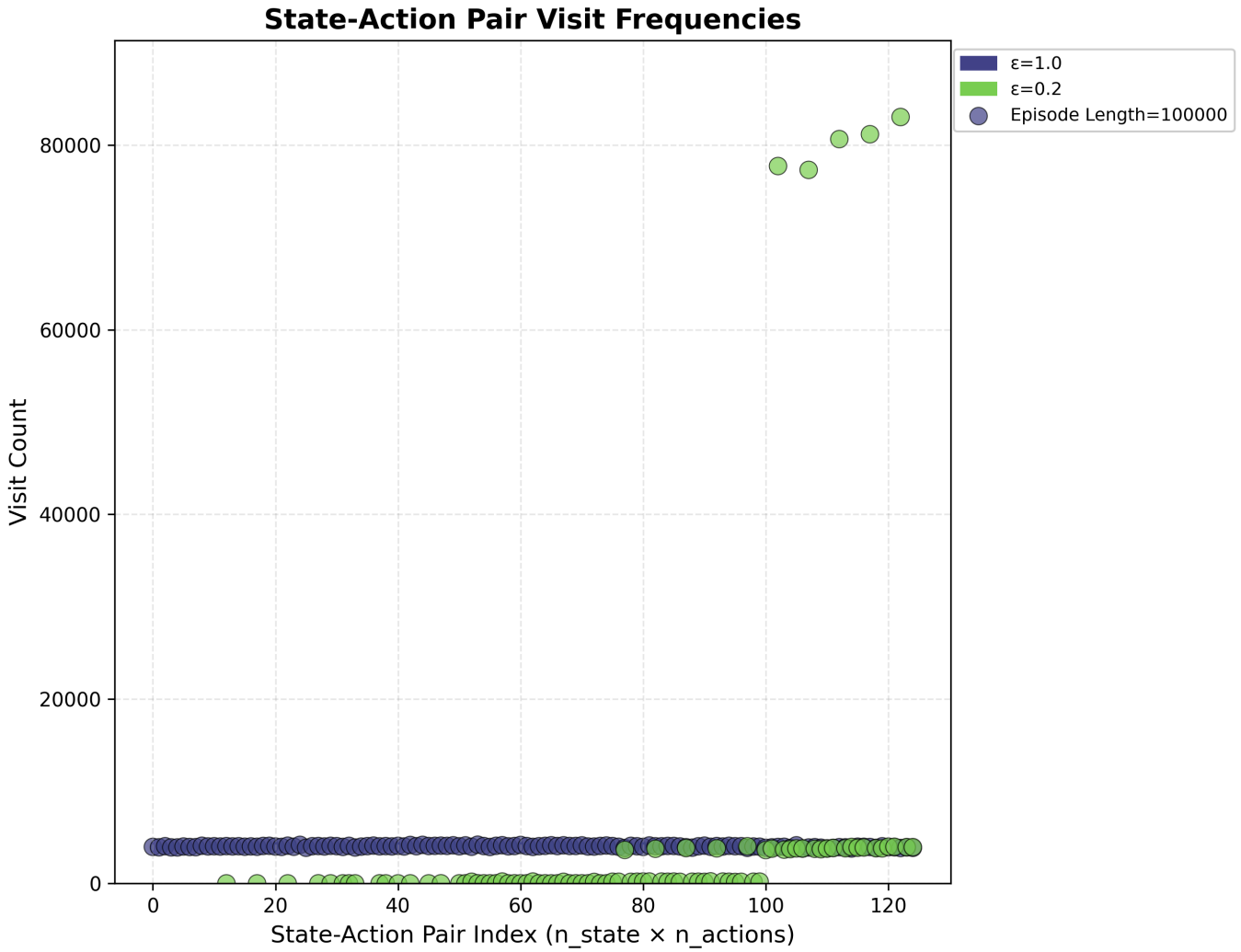
1.75 0 ↓	1.59 1 →	2.04 2 ↓	2.25 3 ↓	1.80 4 ←
2.16 5 ↓	3.03 6 ↓	3.58 7 ↓	2.71 8 ↓	2.45 9 ←
2.71 10 ↓	3.58 11 →	4.88 12 ↓	3.26 13 ↓	3.17 14 ←
3.29 15 →	4.79 16 →	1.00 17 ○	4.88 18 ←	3.57 19 ←
3.06 20 ↑	3.91 21 ↑	3.17 22 →	3.70 23 ↑	3.25 24 ←

MC ε-greedy (ε=0.1)
Final Policy

-0.33 0 →	3.87 1 ↓	2.36 2 ←	1.51 3 ←	-0.18 4 ←
3.04 5 ↓	5.74 6 ↓	0.21 7 →	1.62 8 ↑	-0.02 9 ○
4.86 10 →	6.47 11 →	8.56 12 ↓	0.73 13 ↑	0.78 14 ←
3.56 15 ↑	8.48 16 →	8.56 17 ○	8.26 18 ←	0.00 19 ↑
3.99 20 ↑	7.46 21 →	8.00 22 ↑	4.38 23 ←	-1.78 24 ←

MC ε-greedy (ε=0.2)
Final Policy

0.48 0 ↓	1.43 1 ○	0.88 2 →	0.19 3 ↓	-0.32 4 ↓
1.03 5 ↓	0.54 6 ←	4.56 7 ↓	2.65 8 ←	0.21 9 ←
4.30 10 ↓	3.55 11 ←	7.13 12 ↓	4.68 13 ←	-0.11 14 ↑
4.96 15 →	7.16 16 →	7.22 17 ○	7.17 18 ←	0.82 19 ↑
1.56 20 ↑	5.71 21 →	7.18 22 ↑	5.64 23 ←	3.56 24 ←



策略对比图展示了不同采样策略学到的贪婪策略和值函数。epsilon-greedy 方法因为每一步都保持探索，目标格附近价值更容易被多次传播；Basic MC 和 Exploring Starts 因为轨迹较短，目标格 `stay` 的长期价值没有完全体现出来。

访问散点图说明了探索机制的重要性：如果某些状态动作对很少被访问，它们的价值估计就会不稳定。MC 方法的理论收敛依赖"足够探索"和"足够样本"，这张图正好把这个抽象条件具体化了。

5. Chapter 5：随机近似与优化

5.1 理论知识点

随机近似是很多免模型 RL 算法背后的数学骨架。假设我们想求某个期望目标：

$$\theta^* = \mathbb{E}[X]$$

但无法一次看到整个分布，只能不断拿到随机样本 `x_k`。一种自然更新是：

$$\theta_{k+1} = \theta_k + \alpha_k(X_k - \theta_k)$$

这与 SGD 形式一致。如果目标函数为：

$$J(\theta) = \frac{1}{2}\mathbb{E}\left[\|\theta - X\|^2\right]$$

则梯度为：

$$\nabla J(\theta) = \theta - \mathbb{E}[X]$$

用单个样本近似梯度，就得到：

$$\theta_{k+1} = \theta_k - \alpha_k(\theta_k - X_k)$$

经典随机近似理论常要求学习率满足：

$$\sum_k \alpha_k = \infty, \qquad \sum_k \alpha_k^2 < \infty$$

例如 `alpha_k = 1/k`。这样的条件让算法既能不断吸收新信息，又能让噪声影响逐渐消失。

5.2 项目实现

第 5 章不是 GridWorld，而是一个二维均值估计实验。环境从正方形区域 `[-25,25] x [-25,25]` 采样 1000 个点，真实总体均值为 `[0,0]`。算法从初始点 `[50,50]` 出发，比较：

方法	更新方式
BGD	使用全部样本均值更新
SGD	每次随机抽一个样本更新
MBGD	每次随机抽一个 mini-batch 更新

项目记录的是估计点到真实均值 `[0,0]` 的距离，而不是到经验样本均值的距离。因为有限样本均值不一定正好等于 `[0,0]`，这里的误差同时包含优化误差和统计误差。

5.2.1 这一章实现了什么，为什么要这样做

第 5 章看起来暂时离开了 GridWorld，但它其实是在补 RL 训练的数学底座。MC、TD、Q-learning、Policy Gradient 都不是一次算出答案，而是通过带噪声的样本一步步更新参数。随机近似就是解释这种“带噪声更新为什么可能收敛”的工具。

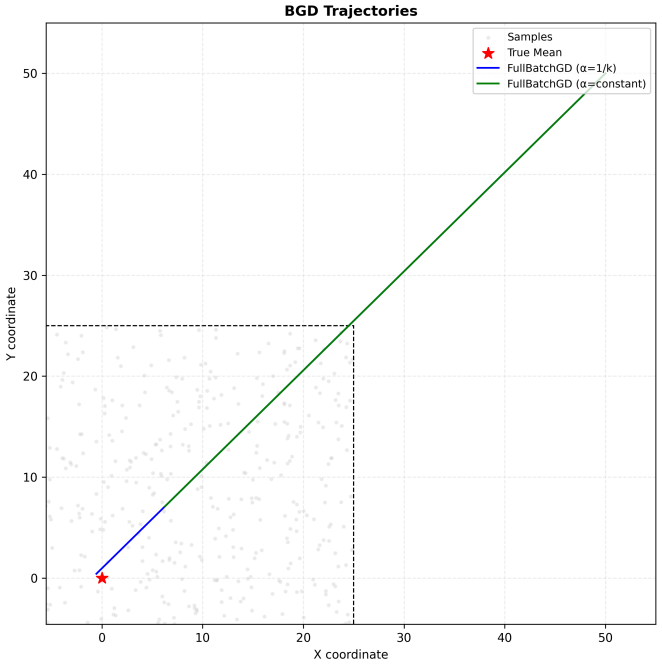
本章用二维均值估计做实验，是因为这个问题足够简单，能把优化现象看得很清楚。真实目标是 `[0,0]`，初始估计是 `[50,50]`，每种算法都在尝试把估计点拉回样本中心。

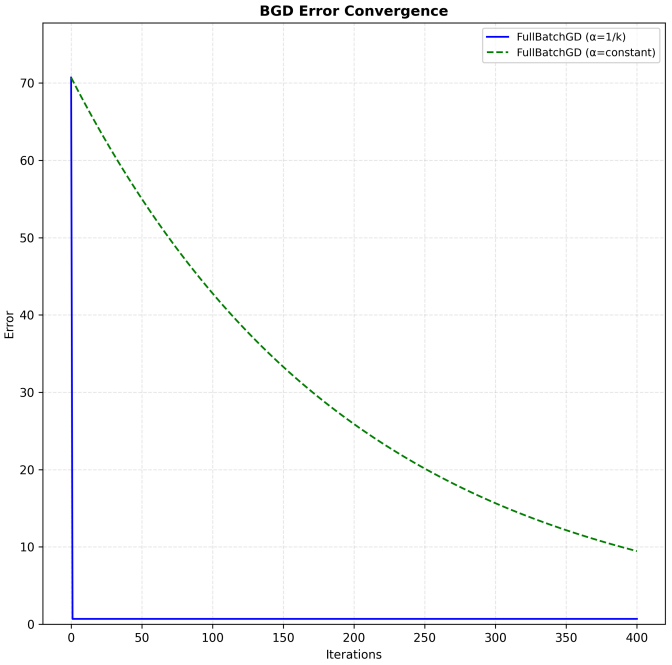
本章三类优化器的作用是：

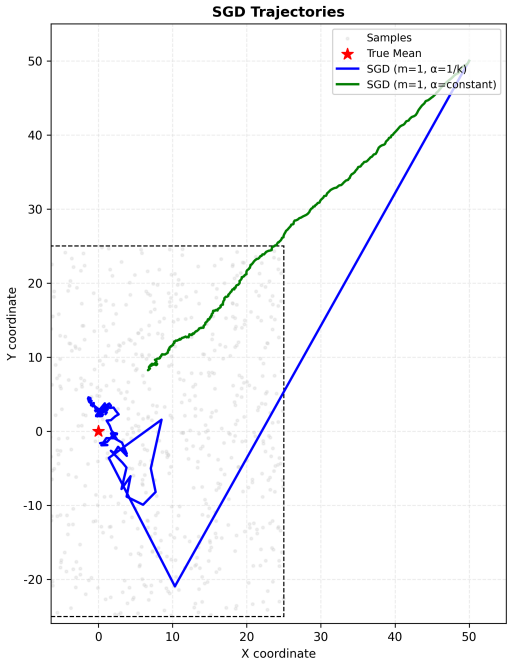
方法	做了什么	为什么做
BGD	每次使用全部样本均值更新	展示低噪声、计算量大的更新
SGD	每次使用单个随机样本更新	展示高噪声、低成本的在线更新
MBGD	每次使用一个小批量更新	展示计算成本和噪声之间的折中

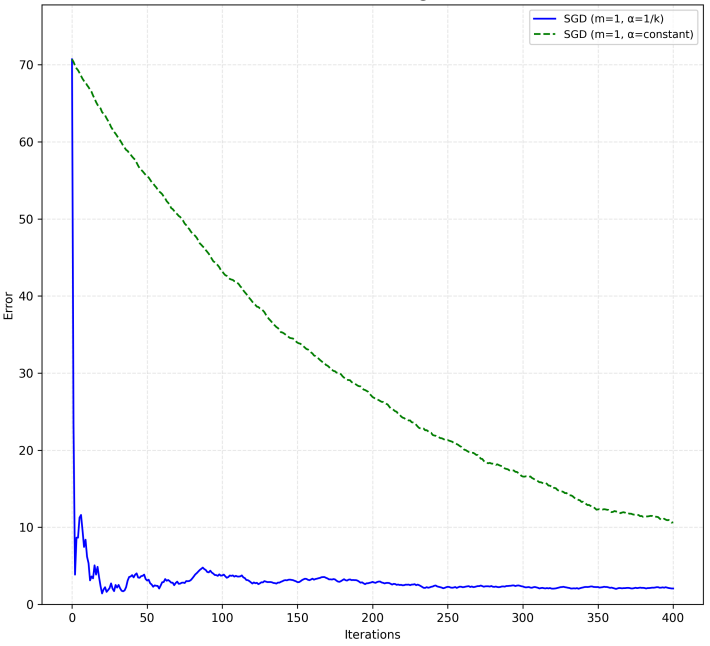
这一章为什么放在 TD 前后都合理？因为 TD 更新本身就像 SGD：它拿到一个 transition，就用这个 transition 构造一个带噪声的目标，然后更新当前估计。理解学习率、噪声、batch size 和收敛曲线，能帮助解释后面为什么有些 RL 曲线会震荡，为什么学习率太大可能不稳定，为什么样本数增加后估计通常会更可靠。

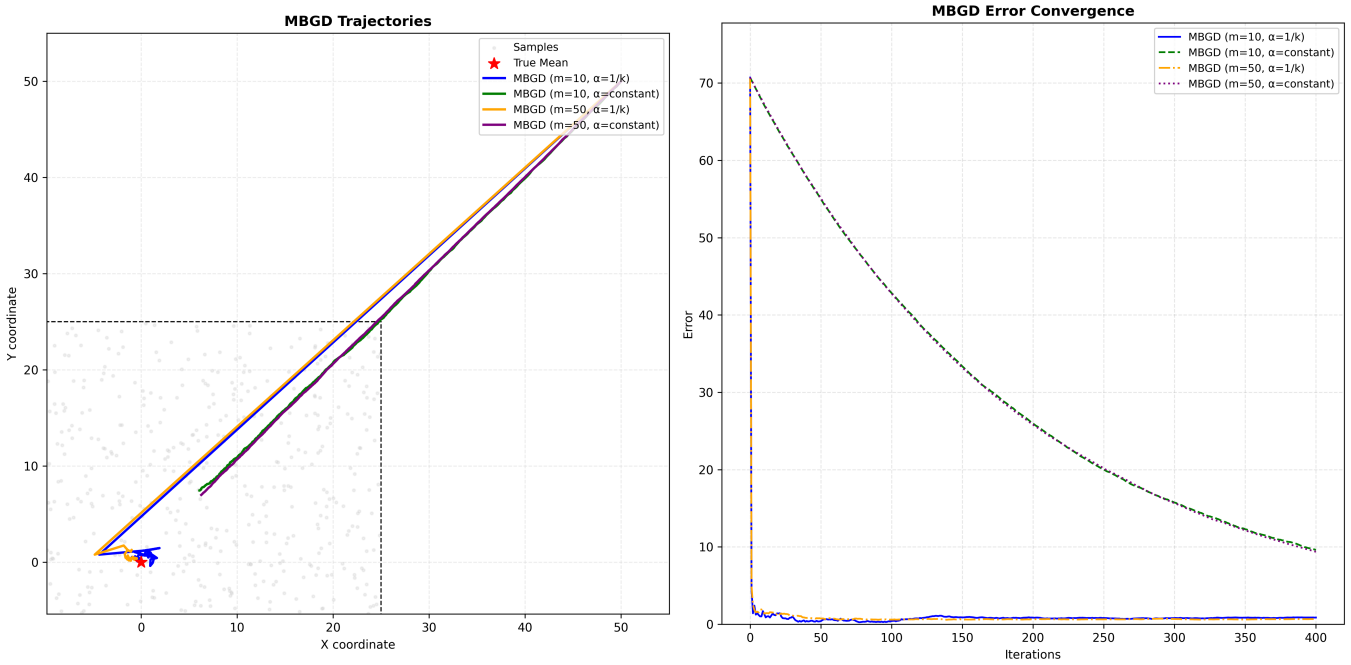
5.3 实验结果解读


BGD Trajectories
Samples
True Mean
FullBatchGD (α=1/k)
FullBatchGD (α=constant)
Y coordinate
X coordinate


BGD Error Convergence
FullBatchGD (α=1/k)
FullBatchGD (α=constant)
Error
Iterations


SGD Trajectories
Samples
True Mean
SGD (m=1, α=1/k)
SGD (m=1, α=constant)
Y coordinate
X coordinate


SGD Error Convergence
SGD (m=1, α=1/k)
SGD (m=1, α=constant)
Error
Iterations



BGD 使用全部样本，因此轨迹平滑；SGD 每次只看一个样本，因此路径噪声大；MBGD 介于二者之间，batch 越大，更新越平滑但每次计算成本越高。

这一章放在 MC 和 TD 之间很自然：MC、SARSA、Q-learning 本质上都是用随机样本更新价值估计。学习率、噪声、方差、batch size 这些优化概念，会直接影响后面 RL 算法的稳定性。

6. Chapter 6：Temporal Difference Learning

6.1 理论知识点

TD Learning 结合了 MC 和 DP 的思想：

- 像 MC 一样，它可以从采样轨迹学习，不需要环境模型；
- 像 DP 一样，它会 bootstrap，用当前价值估计更新当前价值。

最基本的 TD(0) 状态价值更新为：

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \right]$$

括号中的部分称为 TD error：

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

控制问题通常学习动作价值 `Q(s,a)`。SARSA 是 on-policy 方法：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$

Q-learning 使用贪婪目标：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

Expected SARSA 把下一步动作价值替换为策略下的期望：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

n-step SARSA 则在 MC 和 TD(0) 之间插值。`n=1` 时接近一步 TD；`n` 越大，越依赖多步实际回报，方差通常更高但 bias 可能更低。

TD 与 MC 最核心的区别是 bootstrap。MC 的目标来自真实采样回报：

$$\text{target}_{MC} = G_t$$

TD 的目标包含当前模型自己的估计：

$$\text{target}_{TD} = R_{t+1} + \gamma V(S_{t+1})$$

因此 TD 更新更快、更在线，但它的目标会受当前估计误差影响。这个 bias-variance 权衡贯穿后面的函数近似和 Actor-Critic：只要使用 critic 或 bootstrapped target，就会得到更低方差和潜在偏差的组合。

6.2 项目实现

第 6 章实现了：

算法	项目中的目标
SARSA	epsilon-greedy 行为策略下的 on-policy 更新
Expected SARSA	使用当前 epsilon-greedy 策略的期望下一价值
n-step SARSA	比较 `n=1` 与 `n=10`
Online Q-learning	逐步采样、逐步更新的 Q-learning
Offline Q-learning	先收集完整轨迹，再按轨迹顺序更新

这里的 `Online Q-learning` 虽然文件名里带有 OnPolicy，但从更新目标看仍然是 Q-learning：行为策略是 epsilon-greedy，目标使用 `max_a Q(s',a)`，因此属于 off-policy control 思想。`Offline Q-learning` 指的是实现流程上先收集一段轨迹再更新，并不等同于现代语境中只使用固定数据集的 offline RL。

实验每个 episode 长度为 1000 步，不因为到达目标而结束。epsilon 初始为 `0.2`，每轮按 `0.998` 衰减，最低到 `0.01`。

6.2.1 这一章实现了什么，为什么要这样做

第 6 章是项目中表格 model-free control 的核心章节。第 4 章 MC 要等完整轨迹结束后才能更新，而 TD 可以每走一步就更新。这让智能体能更快利用新经验，也更接近在线学习场景。

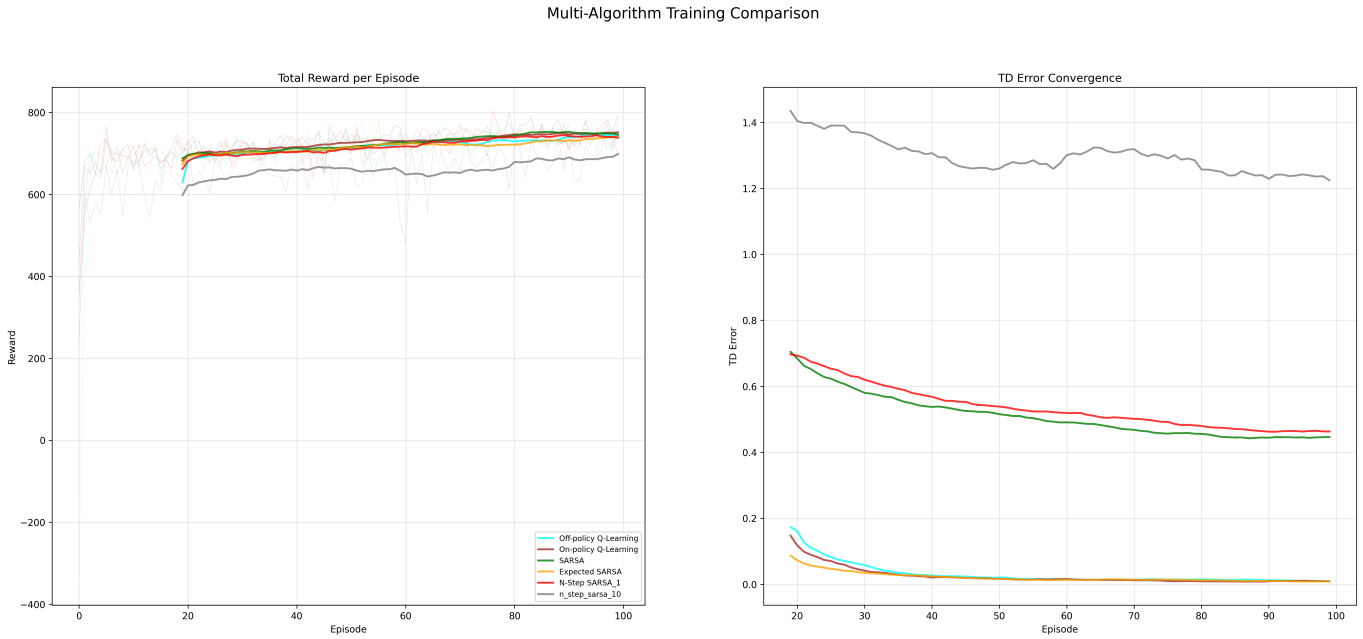
本章做的事情可以分成三层：

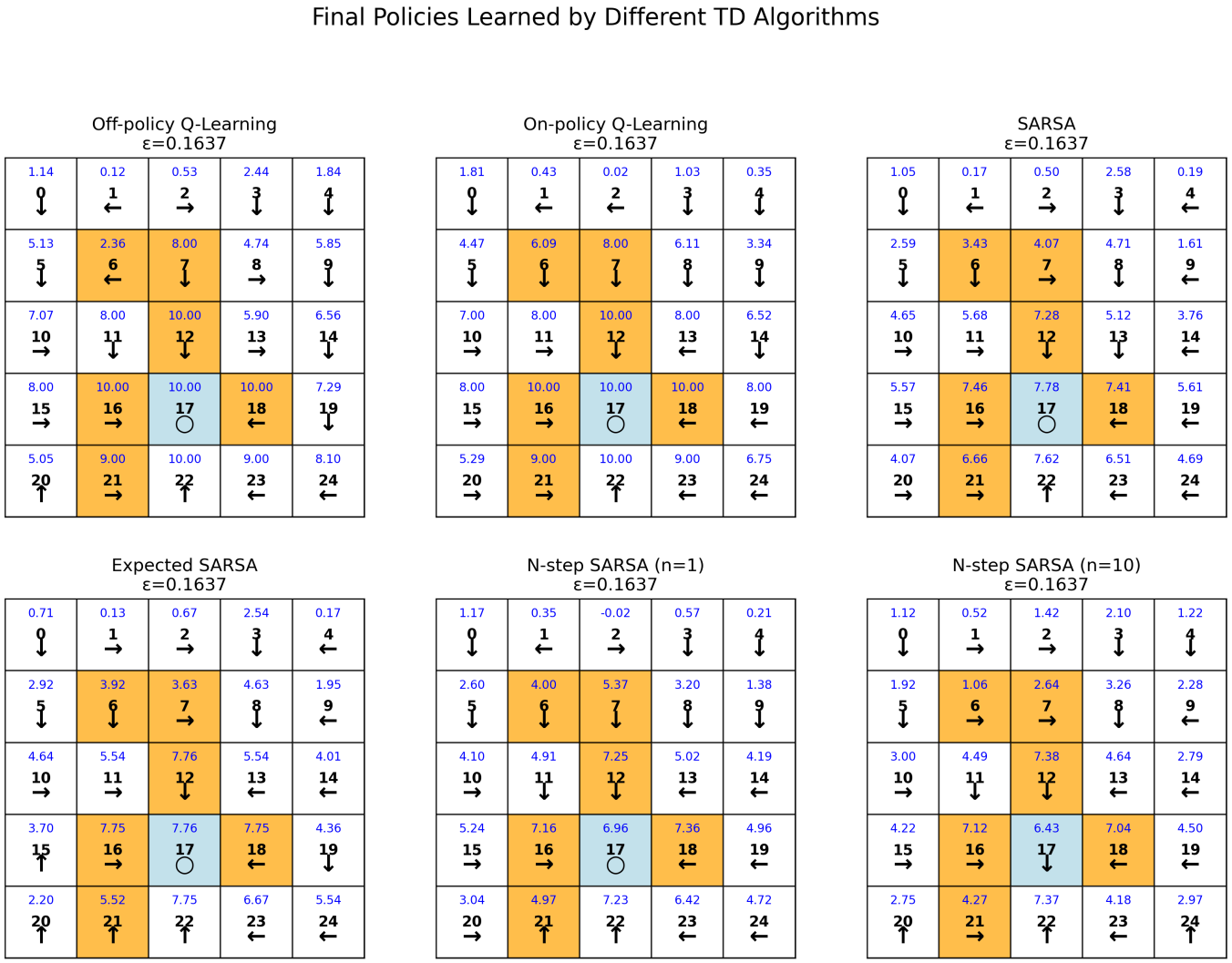
层次	实现内容	目的
一步 TD 控制	SARSA、Expected SARSA、Q-learning	比较不同 TD target
多步 TD 控制	n-step SARSA	观察 MC 和 TD(0) 之间的折中
更新流程对比	online 与先收集再更新的 Q-learning	区分交互方式对学习曲线的影响

SARSA、Expected SARSA 和 Q-learning 最大的区别在下一状态怎么估计。SARSA 使用实际采样到的下一动作，所以更贴近当前行为策略；Expected SARSA 对下一动作求策略期望，降低了采样动作带来的方差；Q-learning 直接取最大动作价值，目标更贪婪，也更接近学习最优策略。

为什么这一章很关键？因为现代很多 RL 算法都还保留着 TD 的核心结构：用即时奖励加上下一状态估计，构造一个 target，然后让当前估计靠近它。DQN、Actor-Critic、PPO 里的 critic 训练，都能看到 TD 思想的影子。这个项目用表格 Q 值先把这个结构展示清楚，再进入第 7 章的函数近似。

6.3 实验结果解读





训练曲线展示了 episode reward 和 TD error 的变化。Q-learning 与 Expected SARSA 的 TD error 更快下降，而 SARSA 和 n-step 方法的误差曲线更高一些。这里不能简单把"TD error 更低"解释成"策略一定更好"，因为不同算法的目标不同、采样路径不同，n-step 目标本身也有不同的尺度。

最终策略图说明，在表格状态动作空间中，TD 方法可以不依赖模型学到可用策略。这一章完成了从 DP 到 model-free control 的过渡：不再需要知道 `P` 和 `R`，只需要与环境交互。

7. Chapter 7：Value Function Approximation

7.1 理论知识点

前几章都使用表格表示，即每个状态或状态动作对都有一个独立参数。但当状态空间很大或连续时，表格法不可行。价值函数近似用参数化函数表示价值：

$$\hat{V}(s;w) \approx V(s), \qquad \hat{Q}(s,a;w) \approx Q(s,a)$$

线性价值函数近似写作：

$$\hat{V}(s;w) = w^\top \phi(s)$$

其中 `phi(s)` 是特征向量。TD(0) 的线性近似更新为：

$$\delta_t = R_{t+1} + \gamma w^\top \phi(S_{t+1}) - w^\top \phi(S_t)$$

$$w \leftarrow w + \alpha \delta_t \phi(S_t)$$

如果是动作价值函数，可以为每个动作维护一组权重：

$$\hat{Q}(s, a; w_a) = w_a^\top \phi(s)$$

函数近似带来了泛化能力：一个状态的更新会通过共享权重影响其他相似状态。但它也带来偏差和不稳定性，尤其当 off-policy、bootstrap 和函数近似同时出现时，训练可能明显波动。

7.1.1 这一章实现了什么，为什么要这样做

第 7 章的目标是回答一个现实问题：如果状态太多，不能再给每个状态单独存一个表格值，怎么办？答案是用函数近似。项目选择多项式特征和线性模型，是一个很适合教学的折中：它比表格法更有泛化能力，又比神经网络更容易解释。

本章分成两个实验：

实验	实现内容	为什么做
实验一	用线性 TD(0) 近似状态价值 `V(s)`	观察不同阶数特征的预测能力
实验二	用线性函数近似做 SARSA 和 Q-learning 控制	观察函数近似下控制算法是否稳定

实验一偏向 prediction：给定采样过程，学习每个状态的价值。实验二偏向 control：价值估计会反过来影响动作选择，动作选择又改变之后看到的数据。因此实验二比实验一更难，也更容易出现波动。

为什么要先做线性函数近似？因为它能清楚展示“特征”这个概念。状态 `(row, col)` 本身只是位置，算法真正看到的是 `phi(s)`。如果特征设计得好，相邻状态会共享相似表示，价值可以平滑传播；如果特征表达不够，或者采样分布和目标价值不匹配，误差就会留下来。这正是从表格 RL 走向深度 RL 必须理解的一步。

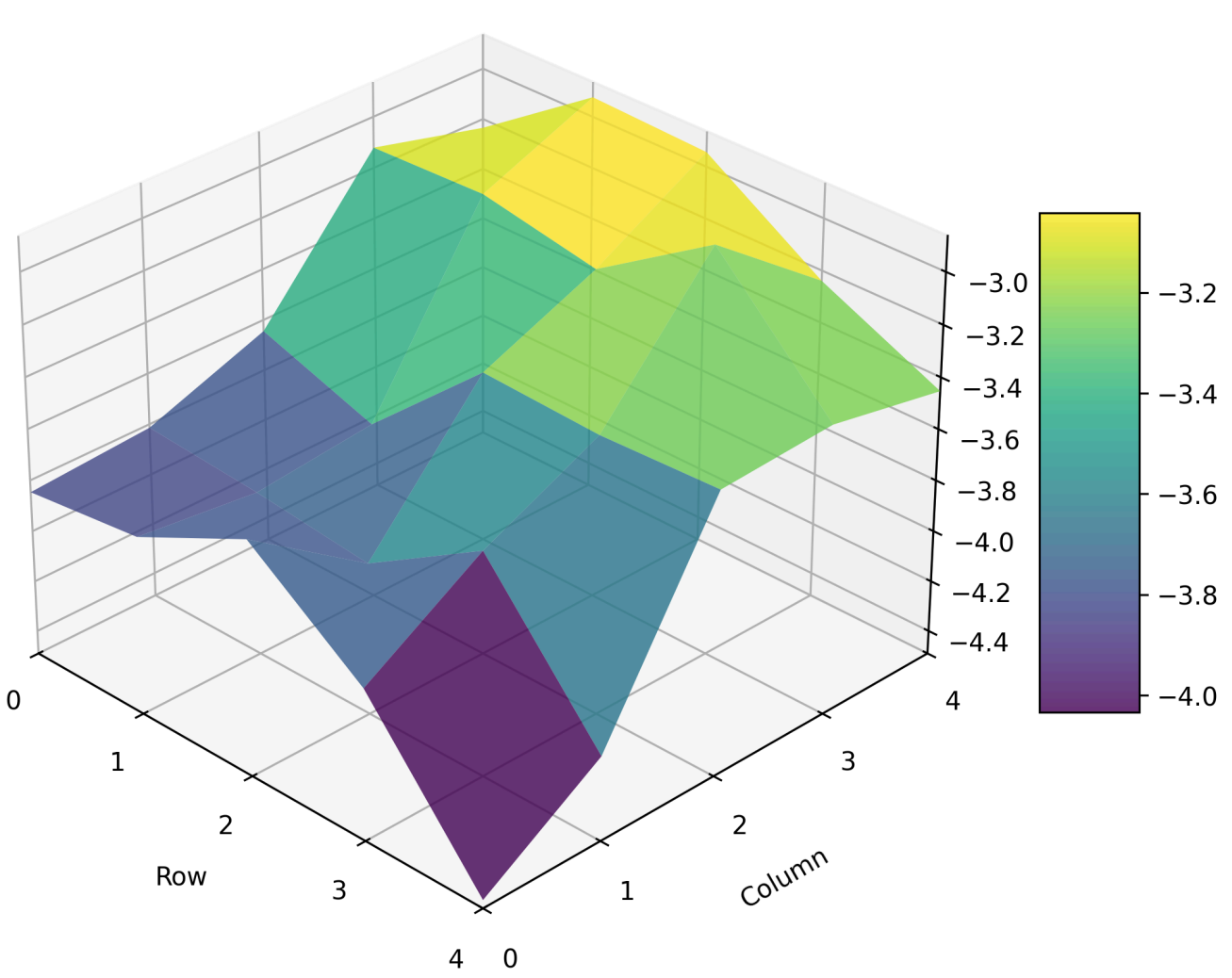
7.2 实验一：线性 TD 预测

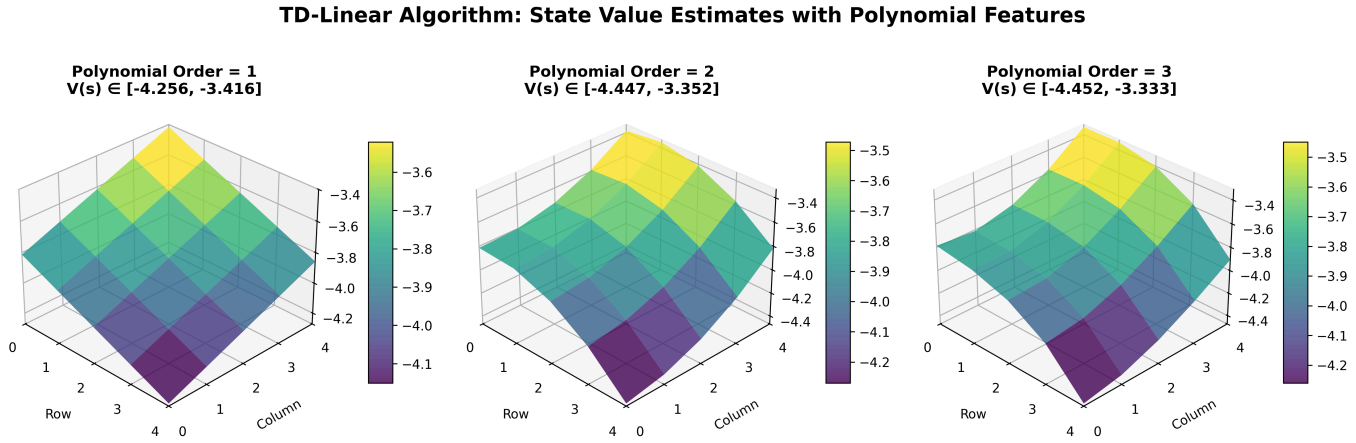
项目使用多项式特征近似状态价值。状态坐标会被归一化到大致 `[-0.5, 0.5]`，然后构造：

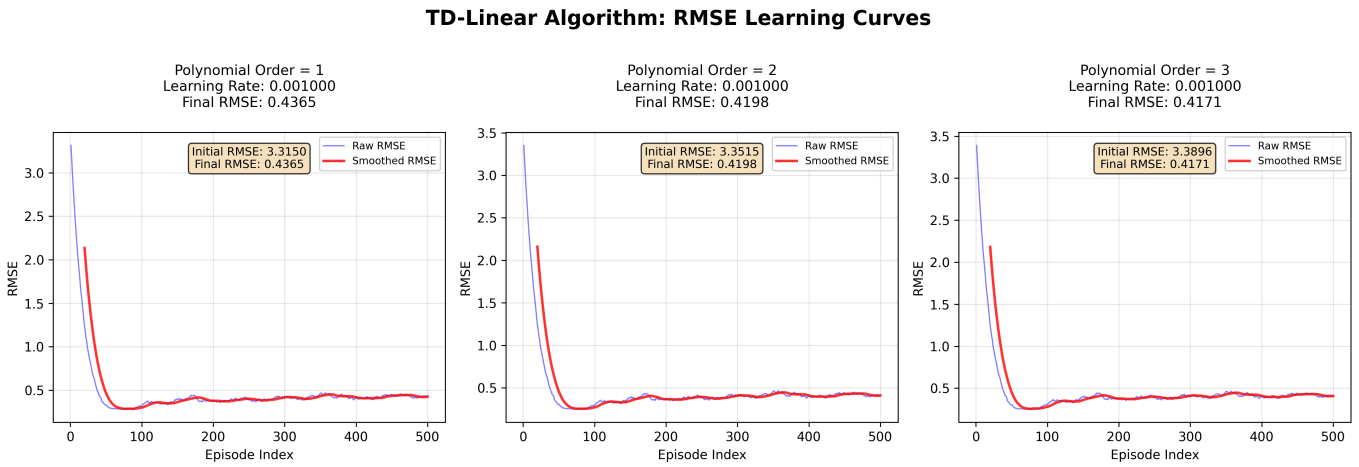
阶数	特征
1 阶	`1, x, y`
2 阶	加入 `x^2, xy, y^2`
3 阶	加入 `x^3, x^2y, xy^2, y^3`

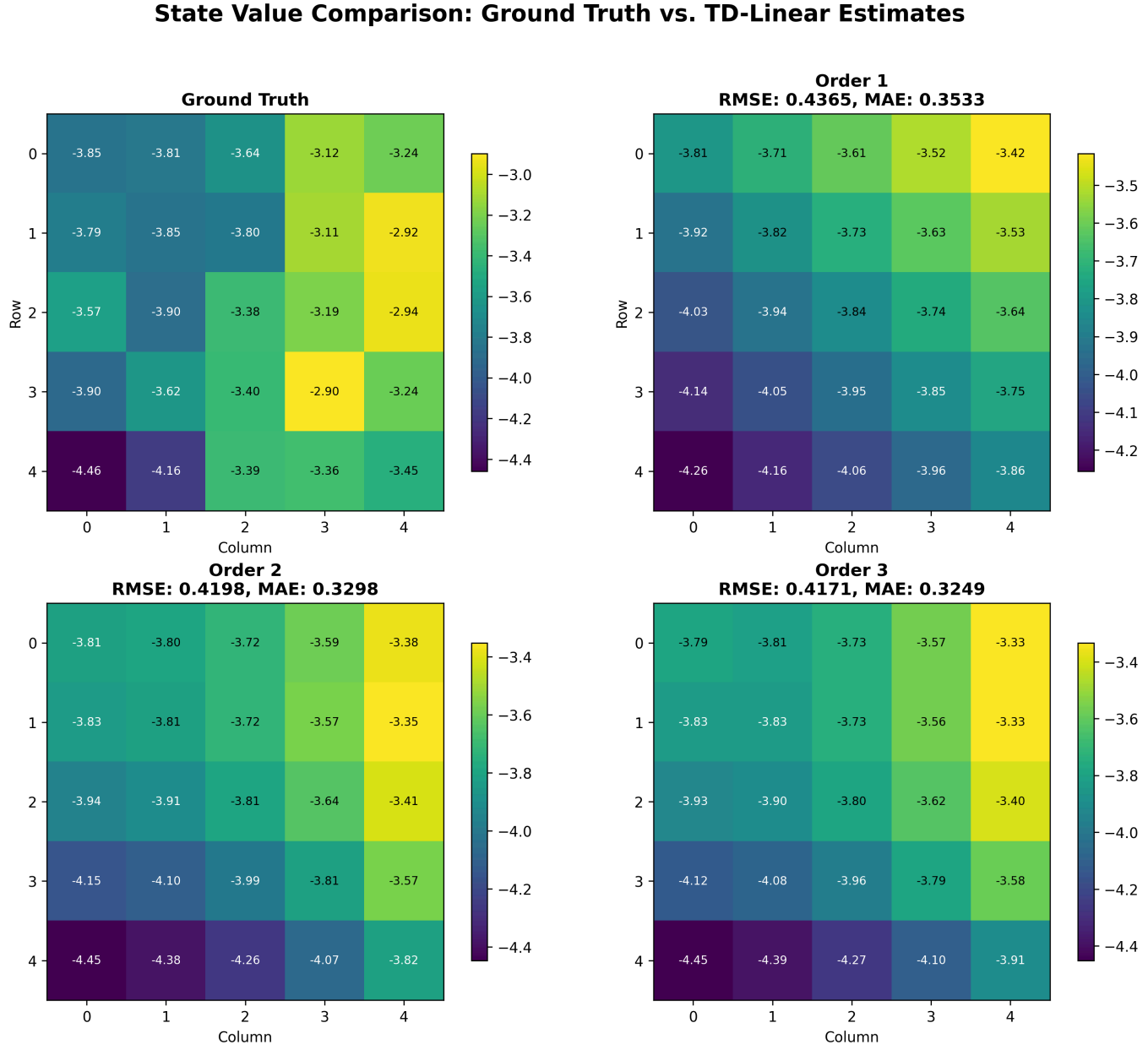
训练使用线性 TD(0)，学习率为 `0.001`，共 500 个 episode，每个 episode 500 步。代码还做了特征范数归一化和权重范数裁剪，这有助于控制数值稳定性。

需要特别说明：仓库 README 中提到 TD(lambda)，但实验一的实际代码是 TD(0)，没有 eligibility trace。另一个影响结果解释的细节是：用于计算参考真值的 Bellman 策略是动作均匀的 `0.2` 策略，而生成训练轨迹的策略是随机归一化后的固定策略。两者不是同一个策略。额外核查显示，这两个参考策略本身的价值 RMSE 约为 `0.4045`，因此图中的误差不能全部归因于函数逼近能力。


Ground Truth - Bellman Iteration Results
Row
Column
0
1
2
3
4
−3.0
−3.2
−3.4
−3.6
−3.8
−4.0
−4.2
−4.4


TD-Linear Algorithm: State Value Estimates with Polynomial Features
Polynomial Order = 1
V(s) ∈ [-4.256, -3.416]
Polynomial Order = 2
V(s) ∈ [-4.447, -3.352]
Polynomial Order = 3
V(s) ∈ [-4.452, -3.333]
Row
Column



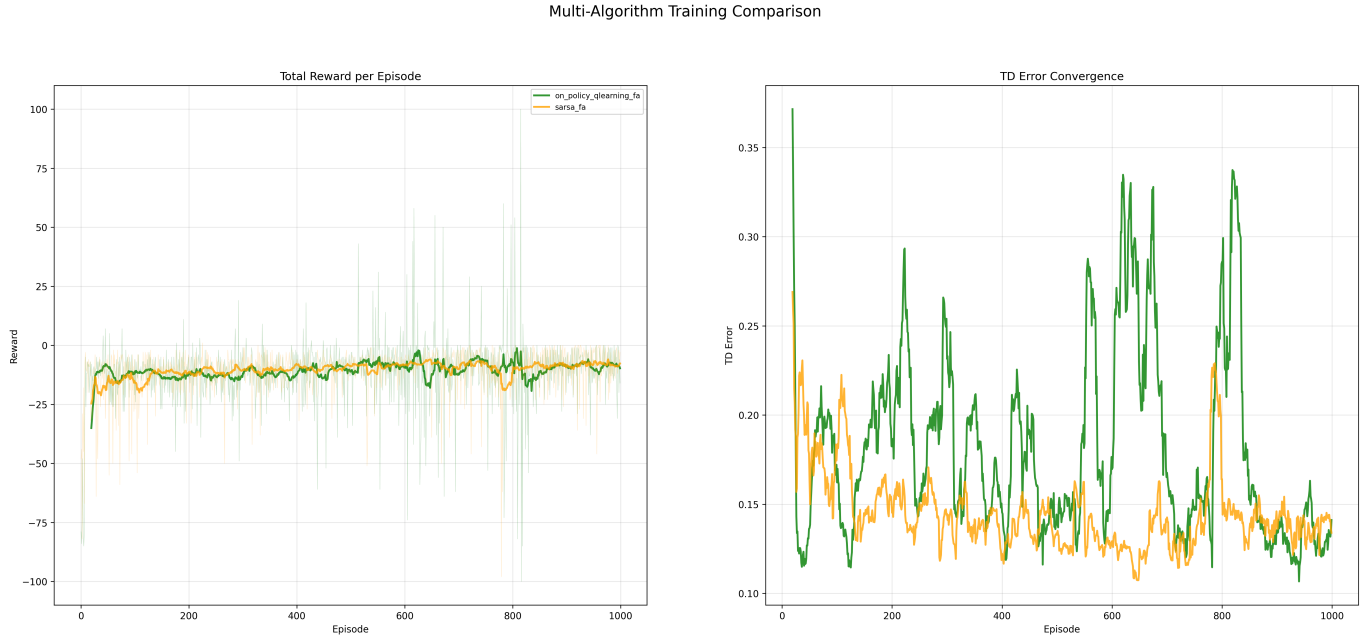


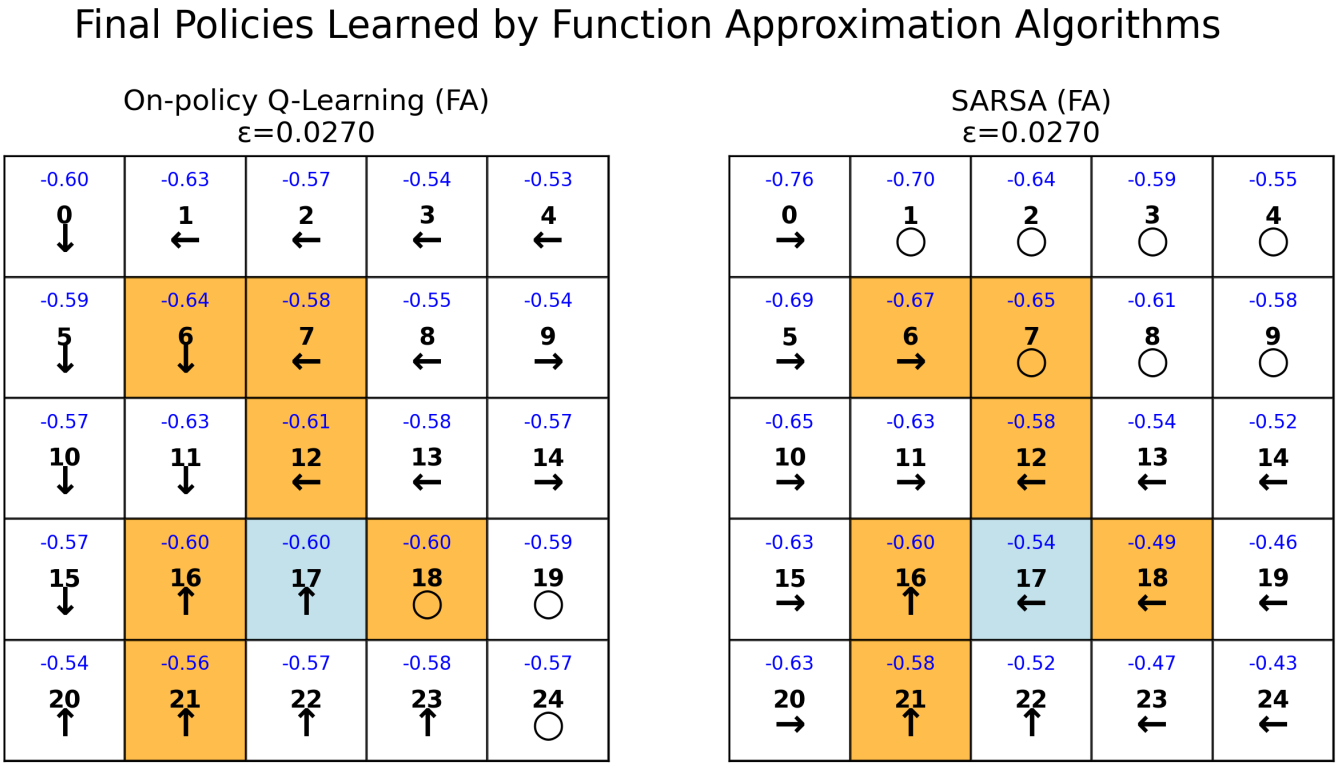
图中 1、2、3 阶特征的最终 RMSE 接近，分别约为 `0.4365`、`0.4198`、`0.4171`。高阶特征略好，但差距不大。曲线先下降后略有回升，说明训练过程存在近似误差、采样策略偏差和 TD bootstrapping 带来的综合影响。

7.3 实验二：函数近似控制

实验二把线性函数近似用于 Q-learning 和 SARSA。默认使用 3 阶多项式特征，并为每个动作维护独立权重。实验参数大致为：

参数	数值
episode 数	1000
每轮最大步数	100
学习率	0.0005
初始 epsilon	0.2
epsilon 衰减	0.998
最小 epsilon	0.01





-0.60 **0** ↓	-0.63 **1** ←	-0.57 **2** ←	-0.54 **3** ←	-0.53 **4** ←
-0.59 **5** ↓	-0.64 **6** ↓	-0.58 **7** ←	-0.55 **8** ←	-0.54 **9** →
-0.57 **10** ↓	-0.63 **11** ↓	-0.61 **12** ←	-0.58 **13** ←	-0.57 **14** →
-0.57 **15** ↓	-0.60 **16** ↑	-0.60 **17** ↑	-0.60 **18** ○	-0.59 **19** ○
-0.54 **20** ↑	-0.56 **21** ↑	-0.57 **22** ↑	-0.58 **23** ↑	-0.57 **24** ○

-0.76 **0** →	-0.70 **1** ○	-0.64 **2** ○	-0.59 **3** ○	-0.55 **4** ○
-0.69 **5** →	-0.67 **6** →	-0.65 **7** ○	-0.61 **8** ○	-0.58 **9** ○
-0.65 **10** →	-0.63 **11** →	-0.58 **12** ←	-0.54 **13** ←	-0.52 **14** ←
-0.63 **15** →	-0.60 **16** ↑	-0.54 **17** ←	-0.49 **18** ←	-0.46 **19** ←
-0.63 **20** →	-0.58 **21** ↑	-0.52 **22** ↑	-0.47 **23** ←	-0.43 **24** ←

从图上看，函数近似控制在当前参数下没有像表格 TD 那样稳定地学到高回报策略。Q-learning 的 TD error 波动更明显，SARSA 相对平滑一些。这不是一个坏结果，反而很适合教学：它展示了从表格法进入函数近似后，算法稳定性不再是理所当然的。

这一章的核心 takeaway 是：函数近似让 RL 可以扩展到更大的状态空间，但也让"表示能力、采样分布、学习率、目标策略、bootstrap"互相纠缠。此时只看单条 reward 曲线往往不够，需要结合误差、策略图、特征设计和多随机种子来判断算法表现。

8. Chapter 8：Policy Gradient

8.1 理论知识点

前面的方法主要先学习价值函数，再通过价值函数导出策略。Policy Gradient 则直接参数化策略：

$$\pi_\theta(a \mid s)$$

目标是最大化期望回报：

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$$

策略梯度定理给出：

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(A_t \mid S_t) G_t\right]$$

REINFORCE 使用完整采样回报估计梯度：

$$\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(A_t \mid S_t)$$

为了降低方差，常引入 baseline：

$$\theta \leftarrow \theta + \alpha \left(G_t - b(S_t)\right) \nabla_\theta \log \pi_\theta(A_t \mid S_t)$$

Actor-Critic 把策略称为 actor，把价值函数称为 critic。critic 估计状态价值，actor 使用 advantage 更新：

$$A_t = G_t - V(S_t)$$

或使用 TD 误差近似：

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

Generalized Advantage Estimation 使用多步 TD 误差的指数加权和，在 bias 和 variance 之间调节。该思想来自 Schulman 等人的 GAE 工作，常用于现代策略梯度算法。

8.2 项目实现

第 8 章使用 PyTorch 实现：

方法	网络
REINFORCE	one-hot 状态输入，MLP 输出动作概率
Actor-Critic	actor 输出动作概率，critic 输出状态价值

这一章的 GridWorld 把目标奖励从前面章节的 `1` 提高到 `10`。每个 episode 最长 20 步，因此单轮未折扣回报的理论上界约为 `200`。

REINFORCE 的实现收集整条轨迹，计算折扣回报，并在每个 episode 内对 returns 做标准化。它没有单独训练一个 value baseline 网络，因此不应把该实现描述为“带价值基线的 REINFORCE”。标准化可以改善数值尺度，但和严格意义上的 learned baseline 不是一回事。

Actor-Critic 使用 actor 和 critic 两个网络，并用类似 GAE 的方式计算 advantage。代码按整段轨迹收集后更新，而不是每一步在线更新。当前实现中 advantage 计算只使用到前 19 个时间步，最后一步 reward/action 没有进入 actor 更新；这会轻微影响训练解释，但不妨碍整体理解 Actor-Critic 的思想。

8.2.1 这一章实现了什么，为什么要这样做

第 8 章从 value-based 方法转向 policy-based 方法。前面的算法大多先学习 `V` 或 `Q`，再通过 `argmax` 得到动作；策略梯度则直接学习动作概率分布 `pi_theta(a|s)`。这对很多任务很重要，尤其是动作空间连续、最优策略本身需要随机性，或者不方便对所有动作取 `argmax` 时。

本章实现了两条路线：

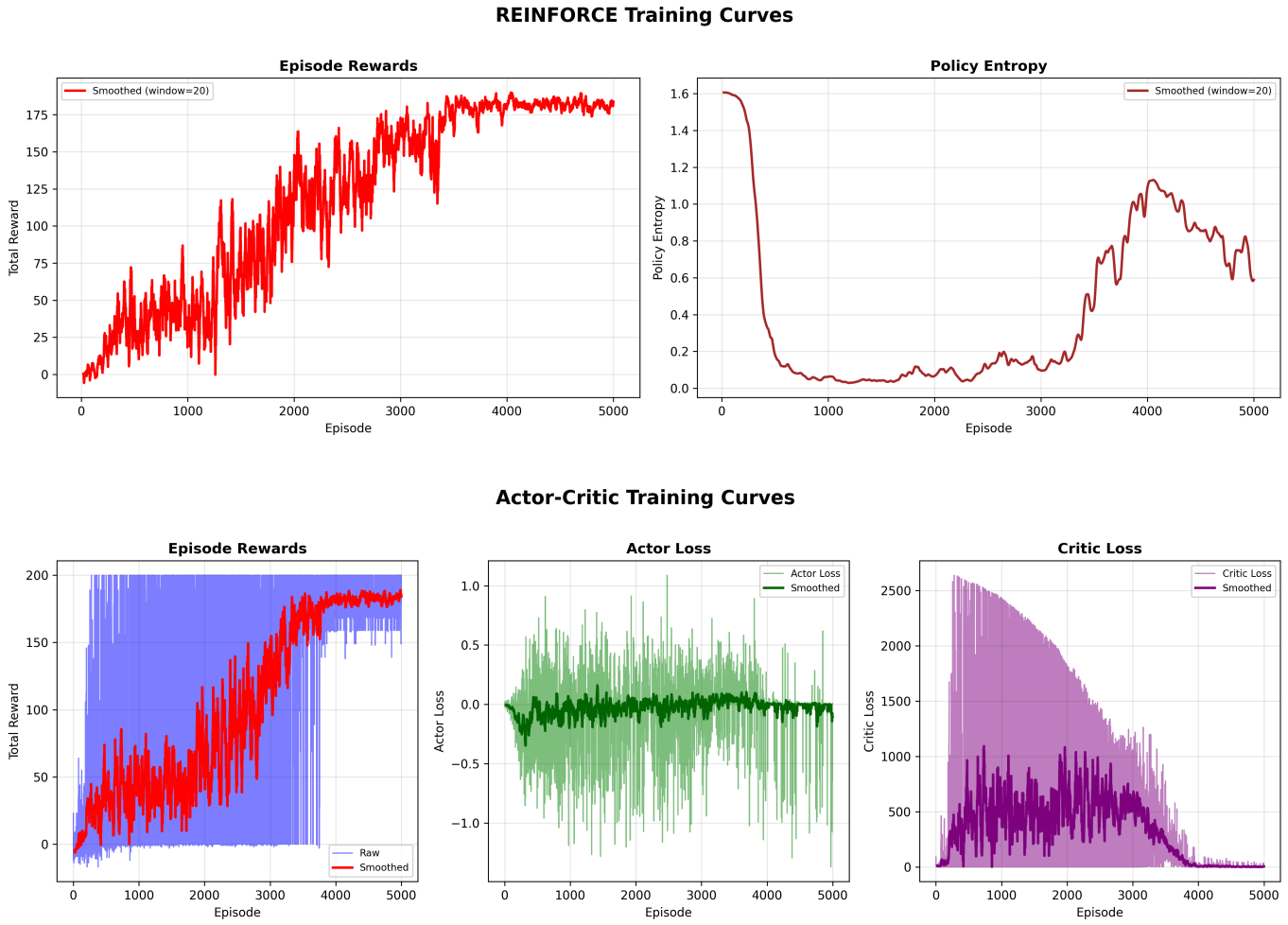
方法	具体做法	为什么做
REINFORCE	用整条 episode 的回报更新策略网络	展示最基础的 Monte Carlo policy gradient
Actor-Critic	同时训练策略网络和价值网络，用 advantage 更新策略	展示 baseline/critic 如何降低策略梯度方差

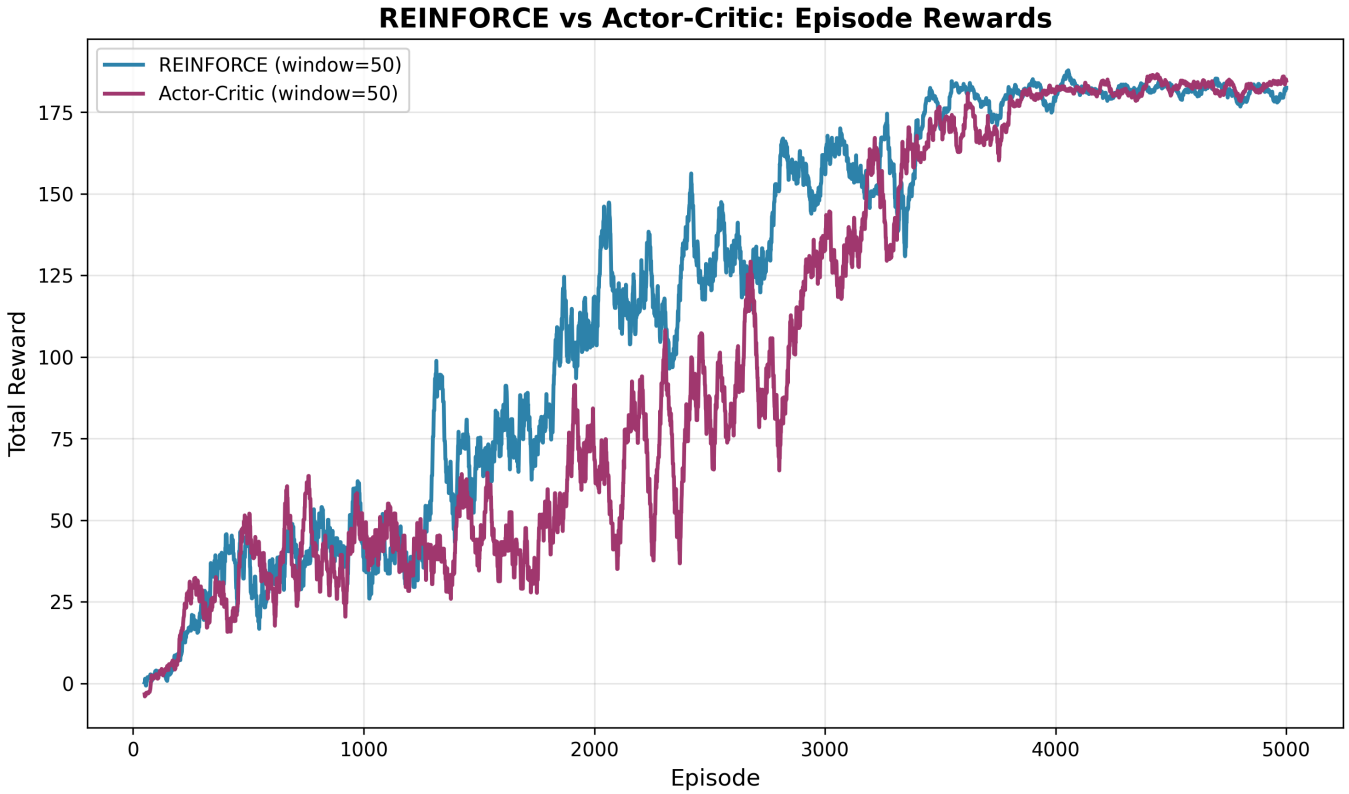
REINFORCE 的逻辑很直接：如果某个动作后面带来了高回报，就提高这个动作在该状态下的概率；如果回报低，就降低概率。它不需要学习 `Q` 表，也不需要对动作做贪婪选择。但代价是方差较大，因为整条轨迹的回报会受到很多随机因素影响。

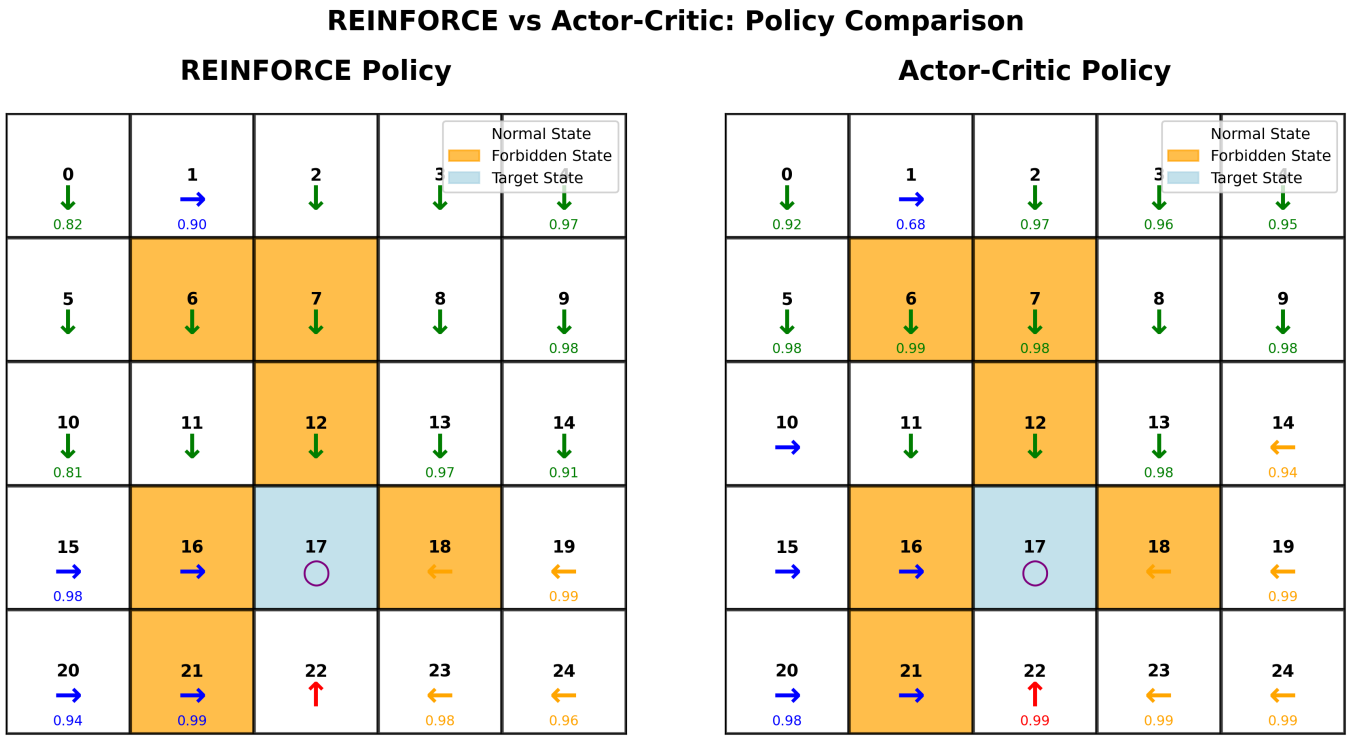
Actor-Critic 引入 critic 后，actor 不再只看“总回报高不高”，而是看“这个动作比当前状态的平均水平好多少”。这就是 advantage 的直觉。如果一个状态本来就容易拿高奖励，那么单纯高回报不一定说明动作特别好；减去 baseline 后，更新信号会更聚焦。

这一章为什么是走向 LLM RL 的桥梁？因为它把前面所有思想合在一起：策略来自概率分布，价值函数用来当 critic，采样轨迹提供训练数据，随机优化更新神经网络参数，advantage 又借用了 TD/GAE 的 bootstrapping 思想。第 9 章会把这些对象逐一对应到 prompt、token、回答、规则奖励和语言模型参数更新。

8.3 实验结果解读







对比图中，REINFORCE 在中期上升较早，后期两者都接近较高回报区间。因为这里只有单次随机种子的平滑曲线，不能据此下强结论说某个算法统计上显著更好。更合理的解释是：在这个小型 GridWorld 中，REINFORCE 和 Actor-Critic 都能学到接近目标格并停留的策略；Actor-Critic 引入 critic 后有更复杂的 bias-variance 权衡，训练表现取决于 critic 质量、学习率、advantage 计算方式和随机种子。

第 8 章把项目从 value-based 方法推进到 policy-based 方法。下一章将这里的 actor、critic、轨迹和 advantage 映射到语言模型的逐 token 生成，并比较需要价值头的 PPO 与不需要价值头的 GRPO。

9. Chapter 9：从传统 RL 到 LLM RL——GSM8K 上的 PPO 与 GRPO

9.1 为什么第 8 章能够通向语言模型强化学习

第 8 章的 Actor-Critic 已经具备现代策略优化的基本部件：actor 给出动作分布，critic 估计状态价值，采样轨迹产生回报，advantage 判断动作相对当前状态基线有多好，第 5 章的随机梯度法负责更新网络。到了语言模型，这些对象没有消失，只是换了一种形式：

传统 GridWorld / Actor-Critic	**GSM8K 语言模型训练**	**联系**
状态 s_t：当前网格位置	题目 prompt 加上已生成的 token 前缀	下一动作取决于当前上下文
动作 a_t：上、下、左、右等	下一个 token	策略输出动作的条件概率
策略 πθ(a_t｜s_t)	Qwen 的下一 token 概率分布	actor 从分布中采样回答
轨迹	从第一个生成 token 到 EOS 的完整回答	一个回答就是一条 episode
环境奖励	从标准答案中提取数值并检查对错	奖励定义优化目标
critic V(s_t)	PPO 中接在隐藏状态上的价值头	估计生成前缀的期望回报
advantage A_t	PPO 的 token 级 GAE；GRPO 的组内相对奖励	决定提高或降低哪些 token 的概率

GridWorld 可以每走一步就得到奖励；GSM8K 的规则奖励主要在一个回答结束后才知道。这是信用分配问题：最终答对或答错，怎样把信号分配给前面生成的 token？PPO 用 critic 与 GAE 将终点奖励向前传播；GRPO 对同一道题生成多个回答，用组内奖励差异构造相对优势。两者都保留了第 8 章“优势加权的策略梯度”这一主干。

9.2 数学任务、奖励和数据流

GSM8K 的每条 JSONL 记录含 question 和 answer。标准答案的最终数值写在 #### 后。代码把题目放入 Qwen 的聊天模板，要求回答也以 #### 数值收尾。src/reward.py 使用正则表达式取出最终数值、去掉千位分隔符，再用 Decimal 比较；与标准答案一致记 1，否则记 0。

```
prompt x  →  逐 token 采样回答 y=(a_1,...,a_T)
          →  提取 #### 后的最终数值
          →  规则奖励 R(x,y) ∈ {0,1}
```

这里的奖励检查最终数值，而非推理过程的每一步。训练集用于策略更新，测试集用于最终答案准确率评估。相对于网格世界的逐步外部反馈，这种终点奖励更稀疏，因此优势估计尤其重要。

9.3 PPO：一个 prompt、一个回答、一个 critic

PPO 入口 src/ppo.py 每次取一道题，在当前策略下采样**一个**回答。模型生成时保存旧策略对各 token 的对数概率；同时保留一份冻结的初始模型作为参考策略。参考策略提供 KL 约束，限制更新后的策略离初始模型过远。对生成序列的有效 token，代码构造：

$$r_t = -\beta \left[\log \pi_{\text{old}}(a_t \mid s_t) - \log \pi_{\text{ref}}(a_t \mid s_t)\right] + \mathbf{1}_{t=T} R(x, y)$$

其中 T 是最后一个有效 token 的位置。非终点 token 主要接收参考策略的 KL 代价，终点 token 再加上数学题的对错奖励。价值头读取语言模型隐藏状态并输出 V(s_t)，即当前生成前缀的预期价值。随后从最后一个 token 逆序计算：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \qquad V(s_{T+1}) = 0$$

$$A_t = \delta_t + \gamma\lambda A_{t+1}, \qquad G_t = A_t + V(s_t)$$

这一步正是第 6 章 TD 误差与第 8 章 Actor-Critic 的延伸。PPO 再比较"当前策略"和"生成回答时的旧策略"，用裁剪目标抑制一次更新中的过大变化，并同时训练价值头：

$$\rho_t = \exp\left[\log \pi_\theta(a_t \mid s_t) - \log \pi_{\text{old}}(a_t \mid s_t)\right]$$

$$L_{\text{PPO}} = -\mathbb{E}_t\left[\min\left(\rho_t A_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)A_t\right)\right] + \frac{c_v}{2}\mathbb{E}_t\left[(V_\theta(s_t) - G_t)^2\right]$$

每个采样回答可做多轮 PPO 更新；mask 只保留首个 EOS 及之前的 token，填充位置不参与损失。PPO 将"critic 预测中间状态 → 终点奖励 → GAE 算每个 token 优势 → 裁剪策略更新"连成了一条信用分配链。

9.4 GRPO：同题多回答，以组内奖励替代 critic

GRPO 入口 src/grpo.py 对同一道题采样一组回答，默认组大小 G=4。每个回答独立计算 0/1 规则奖励。GRPO 不训练价值头，而是用同组平均奖励作为比较基线：

$$\bar{R} = \frac{1}{G}\sum_{i=1}^{G} R_i, \qquad A_i = \frac{R_i - \bar{R}}{\text{std}(R_1, \ldots, R_G) + \epsilon}$$

同一个回答的所有有效 token 共享其组内优势 A_i。比组平均更好的回答得到正优势，其 token 概率倾向于提高；比组平均差的回答得到负优势。当前策略与旧策略的概率比仍做裁剪，并加入相对冻结参考策略的 KL 项；每个回答先按 token 数取平均，再对整个组取平均：

$$\rho_{i,t} = \exp\left[\log \pi_\theta(a_{i,t} \mid s_{i,t}) - \log \pi_{\text{old}}(a_{i,t} \mid s_{i,t})\right]$$

$$L_{\text{GRPO}} = -\mathbb{E}_i\mathbb{E}_t\left[\min\left(\rho_{i,t}A_i, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)A_i\right)\right] + \beta\mathbb{E}_i\mathbb{E}_t\left[\text{KL}_{i,t}\right]$$

$$\Delta_{i,t} = \log \pi_{\text{ref}}(a_{i,t} \mid s_{i,t}) - \log \pi_\theta(a_{i,t} \mid s_{i,t}), \qquad \text{KL}_{i,t} = \exp(\Delta_{i,t}) - \Delta_{i,t} - 1$$

这个设计把第 8 章的"减去 baseline"变成"同题回答互相比较"。若整组奖励完全相同，组内优势为零；这说明 GRPO 的策略信号取决于同题回答的质量差异，而不仅仅是采样数量。

9.5 项目实现、参数与观察方式

文件/模块	完成的事情	为什么这样拆分
scripts/download_gsm8k.py、src/data.py	下载并读取原始训练/测试 JSONL	保持数据格式清楚，不把数据逻辑藏进训练循环
src/reward.py	提取 #### 后的最终答案并计算二值奖励	同一奖励函数供 PPO、GRPO 和评估共用
src/modeling.py	聊天模板、采样、EOS 掩码、token 对数概率、模型保存	让两种算法共享模型接口

文件/模块	完成的事情	为什么这样拆分
src/ppo.py	单回答采样、价值头、GAE、裁剪更新	对照第 8 章 Actor-Critic，展示 token 级信用分配
src/grpo.py	同题多回答、组内标准化优势、裁剪更新	展示不训练 critic 的另一种优势构造方式
src/evaluate.py	在测试集上贪婪生成并统计最终答案准确率	训练和评估使用同一种答案判定规则

默认模型为 Qwen/Qwen2.5-0.5B-Instruct。两个训练入口默认读取前 64 道训练题、更新 20 次、最多生成 128 个 token；PPO 每题采样一个回答，GRPO 每题采样四个回答。章节依赖文件只列出 PyTorch 与 Transformers，数据下载使用标准库。运行入口为：

```
python Chapter9_LLM_RL_GSM8K/scripts/download_gsm8k.py
python -m Chapter9_LLM_RL_GSM8K.src.ppo --steps 20
python -m Chapter9_LLM_RL_GSM8K.src.grpo --steps 20 --group-size 4
python -m Chapter9_LLM_RL_GSM8K.src.evaluate --model
Chapter9_LLM_RL_GSM8K/outputs/grpo
```

观察 PPO 时，重点看单题奖励、策略损失和价值损失如何配合；观察 GRPO 时，重点看组平均奖励与组内标准差。两种方法的共同核心是“采样数据由当前策略产生，再用奖励和优势修改策略”；差别在于 PPO 学习一个中间状态的价值函数，GRPO 使用同题回答之间的相对比较。

10. 从九章看强化学习算法之间的关系

前八章把很多看似分散的算法放在同一个网格环境里；第九章换用 GSM8K，展示这些思想如何迁移。各章之间的关系可以这样理解：

章节	实现算法/模块	做了什么	为什么做	实验重点与注意点
Chapter 1	GridWorld 环境模型	构建 5x5 状态空间、5 个动作、禁止格、目标格、奖励矩阵 `R` 和转移张量 `P`	为第 1 至 8 章的网格实验提供统一 MDP 环境	目标格不是终止状态，`stay` 可持续获得奖励
Chapter 1	随机策略与价值可视化	生成策略矩阵，计算并画出策略箭头和值函数	把策略、价值这些抽象概念变成可观察结果	随机策略下价值不会稳定集中到目标格
Chapter 2	Bellman 闭式求解	解线性方程 $(I-\gamma P_\pi)v=r_\pi$	说明给定策略价值可以被精确求出	小规模有限状态下适用，大规模问题计算成本高
Chapter 2	Bellman 迭代求解	重复应用 `v <- r + gamma Pv`	展示 Bellman backup 的收敛过程	与闭式解最大差约 `9.2e-6`
Chapter 2	多策略价值比较	比较不同手工策略和随机策略的价值图	说明策略改变会直接改变长期回报	随机策略是随机归一化概率，不是严格均匀策略

章节	实现算法/模块	做了什么	为什么做	实验重点与注意点
Chapter 3	Policy Iteration	策略评估到收敛后做贪婪策略改进	实现“评价当前策略，再改进策略”的经典控制闭环	理论清晰，但每轮评估计算较重
Chapter 3	Value Iteration	直接做 Bellman 最优更新并提取策略	更直接地求最优价值和最优策略	与 PI 的最终价值最大差约 `6.84e-6`
Chapter 3	Truncated Policy Iteration	每轮只做固定次数策略评估，再改进	展示评估精度和计算成本的折中	`x` 越大越接近完整策略迭代，`x=1` 误差明显
Chapter 4	Basic Monte Carlo	遍历状态动作对生成短轨迹并估计 `Q(s,a)`	在无模型情况下用实际回报替代 Bellman 期望	固定短轨迹会带来尾部截断偏差
Chapter 4	Exploring Starts MC	随机选择初始 `(s,a)` 采样轨迹	增强状态动作访问覆盖	项目实现更接近“反向去重后的最后访问”更新
Chapter 4	Epsilon-Greedy MC	轨迹内部按 epsilon-greedy 探索和利用	更接近真实控制问题中的采样方式	探索率影响访问分布和价值传播
Chapter 5	Batch Gradient Descent	使用全部样本均值更新二维估计点	展示低噪声但计算量大的优化方式	路径平滑，适合作为对照
Chapter 5	Stochastic Gradient Descent	每次使用单个随机样本更新	展示随机样本带来的噪声和在线更新能力	曲线震荡更明显
Chapter 5	Mini-batch Gradient Descent	每次使用小批量样本更新	说明 batch size 如何折中噪声和成本	是理解后续 RL 训练曲线波动的基础
Chapter 6	SARSA	使用实际下一动作构造 TD target	展示 on-policy TD control	学到的是包含探索行为影响的策略价值
Chapter 6	Expected SARSA	对下一动作按当前策略求期望	降低下一动作采样带来的方差	TD error 下降较快，但仍受当前策略影响
Chapter 6	n-step SARSA	使用多步回报加 bootstrap 更新	连接一步 TD 和 Monte Carlo	`n` 越大越依赖真实多步回报，方差和尺度也会变
Chapter 6	Online Q-learning	逐步采样并用贪婪目标更新	学习接近最优的动作价值函数	文件命名含 OnPolicy，但更新目标本质是 Q-learning
Chapter 6	Offline-style Q-learning	先收集一段轨迹，再按顺序更新	比较更新流程对曲线的影响	这里的 offline 不是现代固定数据集 offline RL

章节	实现算法/模块	做了什么	为什么做	实验重点与注意点
Chapter 7	线性 TD(0) 价值预测	用 1、2、3 阶多项式特征近似 `V(s)`	从表格价值过渡到参数化价值函数	README 提到 TD(lambda)，实际代码为 TD(0)
Chapter 7	多项式特征比较	比较不同阶数特征的 RMSE 和价值曲面	观察特征复杂度对近似能力的影响	三阶略好但差距不大，且参考策略与采样策略不一致
Chapter 7	函数近似 SARSA	用线性 `Q(s,a;w)` 做 on-policy 控制	观察函数近似下控制算法的稳定性	当前参数下回报提升有限
Chapter 7	函数近似 Q-learning	用线性 `Q(s,a;w)` 做 off-policy 控制	展示 off-policy、bootstrap、函数近似组合后的波动	TD error 波动明显
Chapter 8	REINFORCE	用完整 episode 回报更新策略网络	展示最基础的 Monte Carlo policy gradient	使用 returns 标准化，但没有独立 value baseline
Chapter 8	Actor-Critic	同时训练 actor 策略网络和 critic 价值网络	用 advantage 降低策略梯度方差	当前实现近似 GAE，但最后一步没有进入 actor 更新
Chapter 8	策略梯度对比实验	比较 REINFORCE 与 Actor-Critic 的回报曲线和最终策略	总结从 value-based 到 policy-based 的过渡	单随机种子结果不宜做统计显著性结论
Chapter 9	GSM8K 规则奖励	从 #### 后提取最终数值并给出 0/1 奖励	把数学题正确性变成策略优化信号	只奖励最终答案，奖励较稀疏
Chapter 9	PPO	单题采样一个回答，用价值头、GAE 和裁剪目标更新	把第 8 章 Actor-Critic 延伸到 token 级信用分配	同时观察策略损失、价值损失与参考策略 KL
Chapter 9	GRPO	同题采样多个回答，以组内标准化奖励计算优势并更新	用相对比较代替单独训练 critic	整组奖励相同则组内优势为零
Chapter 9	测试集评估	贪婪生成并统计最终答案准确率	用统一规则比较策略输出	评估只加载保存的策略

一条更直观的主线是：

```
MDP 建模
  -> Bellman 方程：价值的递归定义
  -> 动态规划：模型已知时求最优
  -> Monte Carlo：模型未知时用完整回报
  -> TD Learning：模型未知时边走边 bootstrap
  -> 函数近似：状态空间变大后共享参数
  -> 策略梯度：直接优化策略分布
```

```
    -> LLM RL：把回答视为轨迹、token 视为动作
    -> PPO / GRPO：用价值头或组内比较估计优势并更新生成策略
```

前七步构成经典强化学习的学习路径；第九章进一步说明这些概念如何进入语言模型的推理训练。

11. 复现实验建议

仓库根目录可安装完整依赖：

```
pip install -r requirements.txt
```

每章目录下有 `scripts/*.sh`，可在 Linux、macOS、WSL 或 Git Bash 中运行。Windows PowerShell 下如果直接运行 `.sh`，需要换成对应的 Python 命令。

推荐复现实验时按以下顺序：

1. 先运行 Chapter 1 和 Chapter 2，确认 GridWorld、策略矩阵、价值函数可视化正常；
2. 再运行 Chapter 3，对比 PI、VI、TPI 的收敛曲线；
3. 然后运行 Chapter 4 和 Chapter 6，理解 MC 与 TD 在不使用模型时的差异；
4. 单独运行 Chapter 5，把学习率和随机噪声的直觉建立起来；
5. 然后运行 Chapter 7 和 Chapter 8，观察函数近似、策略梯度与 critic；
6. 最后进入 Chapter 9，先读 PPO 的单回答与价值头流程，再读 GRPO 的同题分组比较，按第 9 章说明下载 GSM8K 并运行训练入口。

如果要把项目扩展成更严谨的课程实验报告，可以补充：

改进点	原因
每个算法运行多个随机种子	单条曲线不能说明统计显著性
保存原始 reward/error 日志	方便画均值、方差和置信区间
统一 episode 长度和奖励尺度	便于跨章节横向比较
明确 target 是否终止	持续任务和 episodic 任务价值定义不同
Chapter 7 统一训练策略和参考策略	避免把策略不一致造成的误差归因于函数近似
Chapter 8 修正最后一步 advantage 处理	让 actor 更新覆盖完整轨迹
Chapter 9 比较不同组大小与奖励设计	观察 GRPO 组内优势和策略信号如何变化

12. 总结

本项目从零实现了强化学习的基础算法。前八章用 GridWorld 展示从模型已知到模型未知、从表格表示到函数近似、从价值学习到直接策略优化的路径；第九章切换到 GSM8K，展示这些概念如何服务于语言模型训练。第 1 至 3 章建立 MDP、Bellman 方程与动态规划；第 4 至 6 章通过 Monte Carlo 和 TD 从采样经验中学习；第 7 章引入函数近似；第 8 章实现 REINFORCE 与 Actor-Critic；第 9 章将回答视为轨迹、token 视为动作，分别用 PPO 的 critic/GAE 和 GRPO 的组内比较构造优势。

实验结果整体符合强化学习理论预期：动态规划方法在小规模已知模型中表现稳定；Monte Carlo 方法依赖充分探索和完整回报，容易受到轨迹长度影响；TD 方法利用 bootstrap 提高在线学习能力；函数近似和策略梯度方法更依赖特征、网络、学习率和采样分布。PPO 与 GRPO 进一步说明，同样的策略梯度框架可以通过不同的优势估计方式处理语言模型的终点奖励。项目因此从传统 RL 的可观察网格实验，连贯过渡到 LLM RL 的生成与优化流程。

由于项目代码和本文档都围绕学习与实验展开，难免可能存在理解不严谨、实现细节遗漏或表述不准确之处。如果读者发现代码、公式、实验解释或文档内容中的问题，欢迎提交 issue、PR 或直接指出，我会继续修正和完善。

13. 参考资料

- 项目仓库：https://github.com/tequila28/Foundations-of-RL-Learning
- 项目推荐教材：《强化学习的数学原理》https://github.com/MathFoundationRL/Book-Mathematical-Foundation-of-Reinforcement-Learning
- Sutton, R. S. and Barto, A. G. Reinforcement Learning: An Introduction, 2nd edition. http://incompleteideas.net/book/the-book-2nd.html
- Schulman et al. High-Dimensional Continuous Control Using Generalized Advantage Estimation. https://arxiv.org/abs/1506.02438
- Schulman et al. Proximal Policy Optimization Algorithms. https://arxiv.org/abs/1707.06347
- Shao et al. DeepSeekMath（提出 GRPO）. https://arxiv.org/abs/2402.03300
- GSM8K 原始数据与格式：https://github.com/openai/grade-school-math
- Qwen2.5-0.5B-Instruct 模型卡：https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct